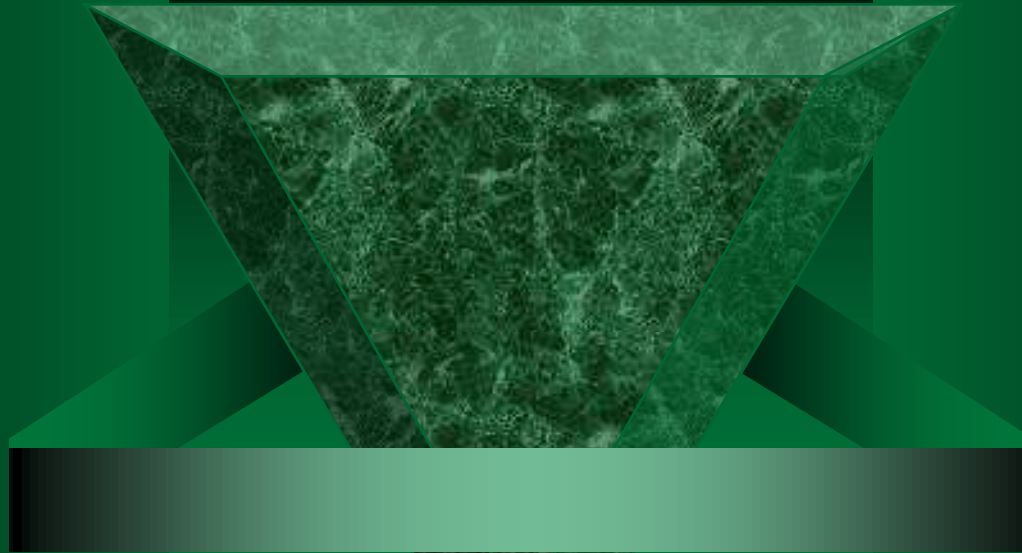


Apirak :



6.1. Function



The Objectives

- ❑ Build structured programs that are divided into function
- ❑ Describe the flow of execution in a program.
- ❑ Describe what the “scope of variable” means?
- ❑ Pass data to functions
- ❑ Get value from function using return

Let consider the app.

```
2 #include <iostream>
3 using namespace std;
4 main()
5 {
6     int choice; // variable for user input
7     int i;       // variable for loops and output
8     do // loop until a valid choice is entered
9     {
10         cout << "Which series do you wish to display?\n";
11         cout << "1 - Odd numbers from 1 to 30\n";
12         cout << "2 - Even numbers from 1 to 30\n";
13         cout << "3 - All numbers from 1 to 30\n";
14         cin >> choice; // get choice from user
15         if ((choice < 1) || (choice > 3))
16         { // if invalid entry, give message
17             cout << "Choice must be 1, 2, or 3\n";
18         }
19     } while ((choice < 1) || (choice > 3));
20     switch (choice)
21     {
22         case 1:
23             for (i = 1; i <= 30; i = i + 2)
24                 cout << i << ' ';
25             cout << endl;
26             break;
27         case 2:
28             for (i = 2; i <= 30; i = i + 2)
29                 cout << i << ' ';
30             cout << endl;
31             break;
32         case 3:
33             for (i = 1; i <= 30; i++)
34                 cout << i << ' ';
35             cout << endl;
36             break;
37     }
38     return 0;
39 }
40
```

ตัวอย่าง 1

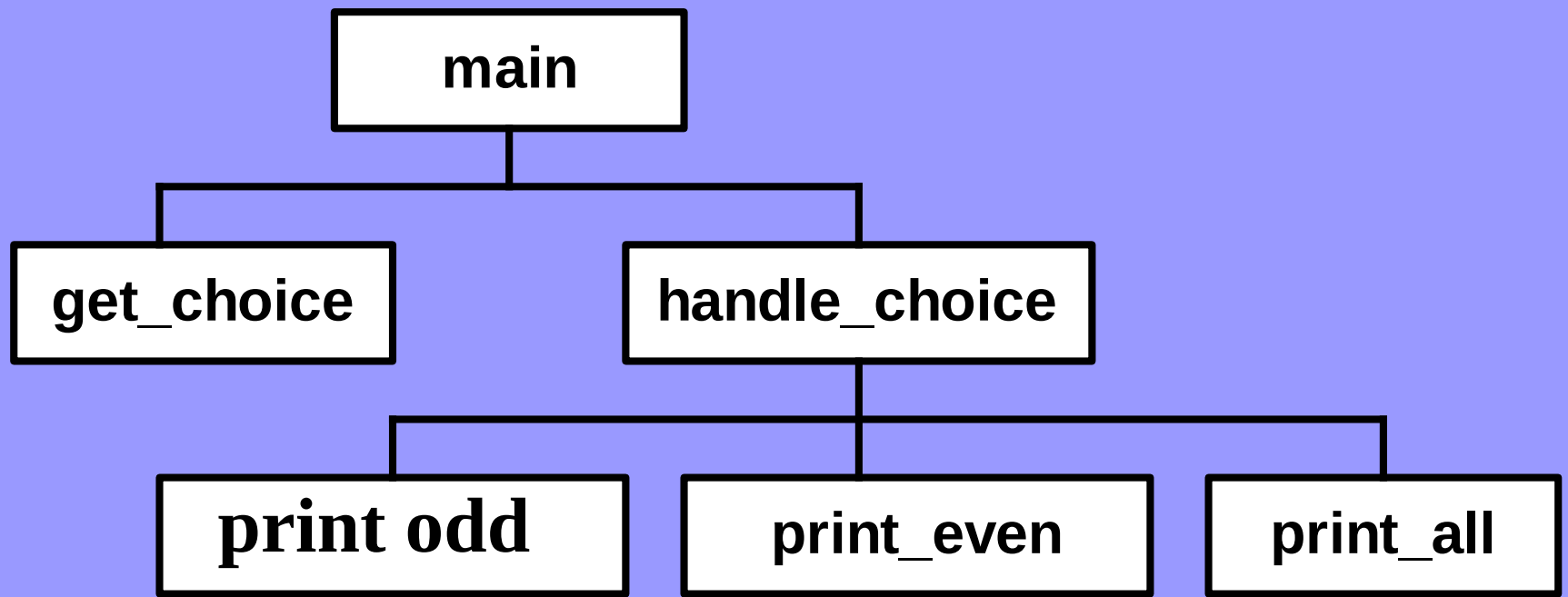
many activities contained in one program (complexity)

โปรแกรมรับข้อมูล 1(พิมพ์เลขคี่),2(พิมพ์เลขคู่),3(พิมพ์ทั้งหมด)

```
C:\My Documents\sc107\files>series
Which series do you wish to display?
1 - Odd numbers from 1 to 30
2 - Even numbers from 1 to 30
3 - All numbers from 1 to 30
1
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29
.....
Which series do you wish to display?
1 - Odd numbers from 1 to 30
2 - Even numbers from 1 to 30
3 - All numbers from 1 to 30
2
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30
```

```
#include<iostream.h> //example1_1.cpp
main()
{
    int choice; // variable for user input
    int i;      // variable for loops and output
    do // loop until a valid choice is entered
    {
        cout << "Which series do you wish to display?\n";
        cout << "1 - Odd numbers from 1 to 30\n";
        cout << "2 - Even numbers from 1 to 30\n";
        cout << "3 - All numbers from 1 to 30\n";
        cin >> choice; // get choice from user
        if ((choice < 1) || (choice > 3))
        { // if invalid entry, give message
            cout << "Choice must be 1, 2, or 3\n";
        }
    } while ((choice < 1) || (choice > 3));
}
```

```
switch (choice)
{
    case 1:
        for (i = 1; i <= 30; i = i + 2)
            cout << i << ' ';
        cout << endl;
        break;
    case 2:
        for (i = 2; i <= 30; i = i + 2)
            cout << i << ' ';
        cout << endl;
        break;
    case 3:
        for (i = 1; i <= 30; i++)
            cout << i << ' ';
        cout << endl;
        break;
}
return 0;
}
```



- *This example if we divide our program to sub-activities like above diagram. it make easier.*
- *C++ supports the idea of dividing program to sub-activities, called **function**.*

ตัวอย่าง 2

Duplicate activity

ให้นักศึกษาเขียนโปรแกรมขายสินค้าและพิมพ์ใบเสร็จรับเงิน

โดยข้อมูลในระบบมีไม่เกิน 100 รายการ

ข้อมูลขายสินค้าด้วย

- รหัสสินค้า(id) เช่น “AB001”
- ชื่อสินค้า(name) เช่น “CAFÉ”
- ราคาขาย(cost) เช่น 30.6
- จำนวนสินค้าในสต็อก (instock) เช่น 5

Menu

1. Add item to Cart
2. Sell Product
3. Check Stock
4. End Program

Please select 1 / 2 / 3/ 4 =?

EX1 จากตัวอย่างข้างต้นให้ นักศึกษาปรับโปรแกรม โดยให้มีการตรวจดังนี้

กรณี กค 1 : ให้ตรวจสอบว่ารหัสที่ป้อน มีซ้ำหรือไม่ ถ้าไม่มีจึงจะอนุญาตให้ผู้ใช้ป้อนข้อมูล

กรณี กค 2: ให้ตรวจสอบว่ารหัสสินค้าที่ผู้ใช้ป้อนมีอยู่ในระบบหรือไม่ กรณีไม่พบให้แสดงข้อความ “product not exist”

กรณี กค 3 ให้แสดงรายการทั้งหมด และแสดงยอดรวมของ cost

Check existing products activity

Apirak :

แบบฝึกหัด 2

จงเขียนโปรแกรมต้องการนำทะเบียนลูกค้า(customer)
ของสหกรณ์ออมทรัพย์รามคำแหง เก็บในตัวแปร
แถวลำดับ 1 มิติ ทะเบียนลูกค้าประกอบด้วย รหัสบัญชี(id)
ชื่อบัญชี(name) ที่อยู่(Address) และเงินฝาก
(Deposit)

เลือก 1 เป็นการเปิดบัญชีใหม่

ระบบจะแสดงหน้าจอให้ผู้ใส่ ป้อนรหัสบัญชี(id)
ชื่อบัญชี(name) ที่อยู่(Address) และเงินฝาก
(Deposit) โดยรหัสบัญชีต้องไม่ซ้ำกัน

เลือก 2 เป็นการฝากเงิน

โดยผู้ใช้ป้อนรหัสบัญชี(id)ที่ต้องการ ระบบจะ
ตรวจสอบเพื่อค้นหาทะเบียนลูกค้า เพื่อให้ผู้ใช้งานป้อน
ยอดฝาก เพื่อปรับปรุงบัญชีให้ถูกต้อง

Menu

1. Open a new Account
2. Deposit
3. Withdraw
4. Compound interest
5. Quit

Please select 1 / 2 / 3 / 4 / 5 =?

เลือก 3 เป็นการถอนเงิน

โดยผู้ใช้ป้อนรหัสบัญชี(id)ที่ต้องการ ระบบจะตรวจสอบเพื่อ
ค้นหาทะเบียนลูกค้า เพื่อให้ผู้ใช้งานป้อนยอดถอนแต่ต้องมีเงินติด
บัญชีอย่างน้อย 100 บาท ระบบทำการปรับปรุงบัญชีให้ถูกต้อง

เลือก 4 เป็นการคิดดอกเบี้ยทบต้น

โดยระบบจะให้ผู้ใช้ป้อนอัตราดอกเบี้ยต่อปี(rate) ทาง
เป็นพิมพ์ เพื่อคิดดอกเบี้ยทบต้นของทุกบัญชี ซึ่งกระทำทุกวันได้
ดอกเบี้ยเป็นรายวัน โปรแกรมจะแสดงเงินฝากเก่า ดอกเบี้ยที่
ได้รับ และเงินฝากใหม่ปรากฏบนจอภาพ

เลือก 5 เป็นการจบการทำงานซ้ำ

```

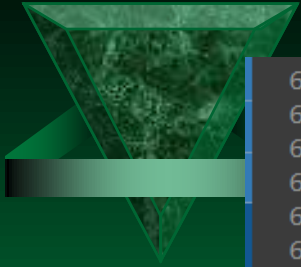
3  using namespace std;
4
5  struct Account {
6      char id[10];
7      char name[100];
8      char addr[100];
9      float deposit;
10 };
11
12 int main()
13 {
14     struct Account cust[500];
15     int n = 0;
16     int choice;
17     char id[10];
18     bool bfound=false;
19     float deposit;
20     int i;
21     do
22     {
23         cout << "\n1.Open Acc";
24         cout << "\n2.Deposit Acc";
25         cout << "\n3.Withdraw Acc";
26         cout << "\n4.Compute Interest";
27         cout << "\n5.exit\n";
28
29         cin >> choice;
30

```

```

31         switch(choice)
32         {
33             case 1:
34                 cout << "\n do Open Acc";
35                 cout << "\n id Acc =? ";
36                 cin >> id;
37                 bfound=false;
38                 for(int i=0 ; i< n ; i++)
39                 {
40                     if( strcmp(id, cust[i].id)==0)
41                     {
42                         cout << "id Acc found =? " << id;
43                         bfound = true;
44                         break;
45                     }
46                 }
47                 if(bfound==false)
48                 {
49                     strcpy( cust[n].id,id );
50                     cout << "name Acc =? ";
51                     cin >> cust[n].name;
52                     cout << "addr Acc =? ";
53                     cin >> cust[n].addr;
54                     cout << "deposit Acc =? ";
55                     cin >> cust[n].deposit;
56                     n++;
57                 }
58
59                 break;

```



```
60         case 2:
61             {
62                 cout << "\n do Deposit  Acc";
63                 cout << "\n id Acc =? ";
64                 cin >> id;
65                 for( i=0 ; i< n ; i++)
66                 {
67                     if( strcmp(id, cust[i].id)==0)
68                     {
69                         cout << "before deposit Acc =? " << cust[i].deposit ;
70                         cout << "\nnew deposit Acc =? ";
71                         cin >> deposit;
72                         cust[i].deposit += deposit;
73                         cout << "\nafter deposit Acc =? " << cust[i].deposit;
74                         break;
75                     }
76                 }
77                 if(i==n) {
78                     cout << "id Acc not found =? " << id;
79                 }
80             }
81             break;
82         case 3:
83             cout << "\n do Withdraw Acc";
84             break;
85         case 4:
86             cout << "\n do Interest Acc";
87             break;
88     } //switch(choice)
89
90     } while(choice!=5);
91
92     return 0;
93 }
```

Rewrite code :Case 2

```
60 case 2:
61 {
62     cout << "\n do Deposit  Acc";
63     cout << "\n id Acc =? ";
64     cin >> id;
65
66     bfound=false;
67     for(int i=0 ; i< n ; i++)
68     {
69         if( strcmp(id, cust[i].id)==0)
70         {
71             cout << "id Acc found =? " << id;
72             bfound = true;
73             break;
74         }
75     }
76     if(bfound==false)
77     {
78         cout << "id Acc not found =? " << id;
79     }
80     else
81     {
82         cout << "before deposit Acc =? " << cust[i].deposit ;
83         cout << "\nnew deposit Acc =? ";
84         cin >> deposit;
85         cust[i].deposit += deposit;
86         cout << "\nafter deposit Acc =? " << cust[i].deposit;
87     }
88 }
89
90 break;
```

EX1

จากตัวอย่างข้างต้นให้ นักศึกษาปรับโปรแกรม โดยให้มีการ
ตรวจดังนี้

กรณี กค 1 : ให้ตรวจสอบว่ารหัสที่ป้อน มีซ้ำหรือไม่ ถ้าไม่มีจึงจะ
อนุญาตให้ผู้ใช้ป้อนข้อมูล

กรณี กค 2: ให้ตรวจสอบว่าสินค้าที่ผู้ใช้ป้อนมีอยู่ในระบบหรือไม่
กรณีไม่พบให้แสดงข้อความ “product not exist”

กรณี กค 3 ให้แสดงรายการทั้งหมด และแสดงยอดรวมของ cost

product List:

Id	name	cost	stock
----	------	------	-------

-------	--	--	--

AB001	CAFÉ	30.6	3
-------	------	------	---

Guidelines for building a program with function

- **Organization:** *A large program is easier to read and modify if it is logically organized into function. Once a single function is tested and performs properly, you can set it aside and concentrate on problem areas.*
- **Autonomy:** *the function does not depend on data or code outside the function any more than necessary.*



Type of Functions(routines) in Programming lang.

□ Library function

- standard library (build-in): coming with programming lang. such as setw(-) , printf(-) in c/c++*
- Third-party library : coding by other programmer teams such buying , opensource.*

□ Customize function : coding by yourself such as main(-) in c/c++

program organization on c/c++ :

```
#include <stdio.h>

int addNumbers(int a, int b);

int main()
{
    ... ..
    sum = addNumbers(n1, n2);
    ... ..
}

int addNumbers(int a, int b)
{
    ... ..
    return result;
}
```

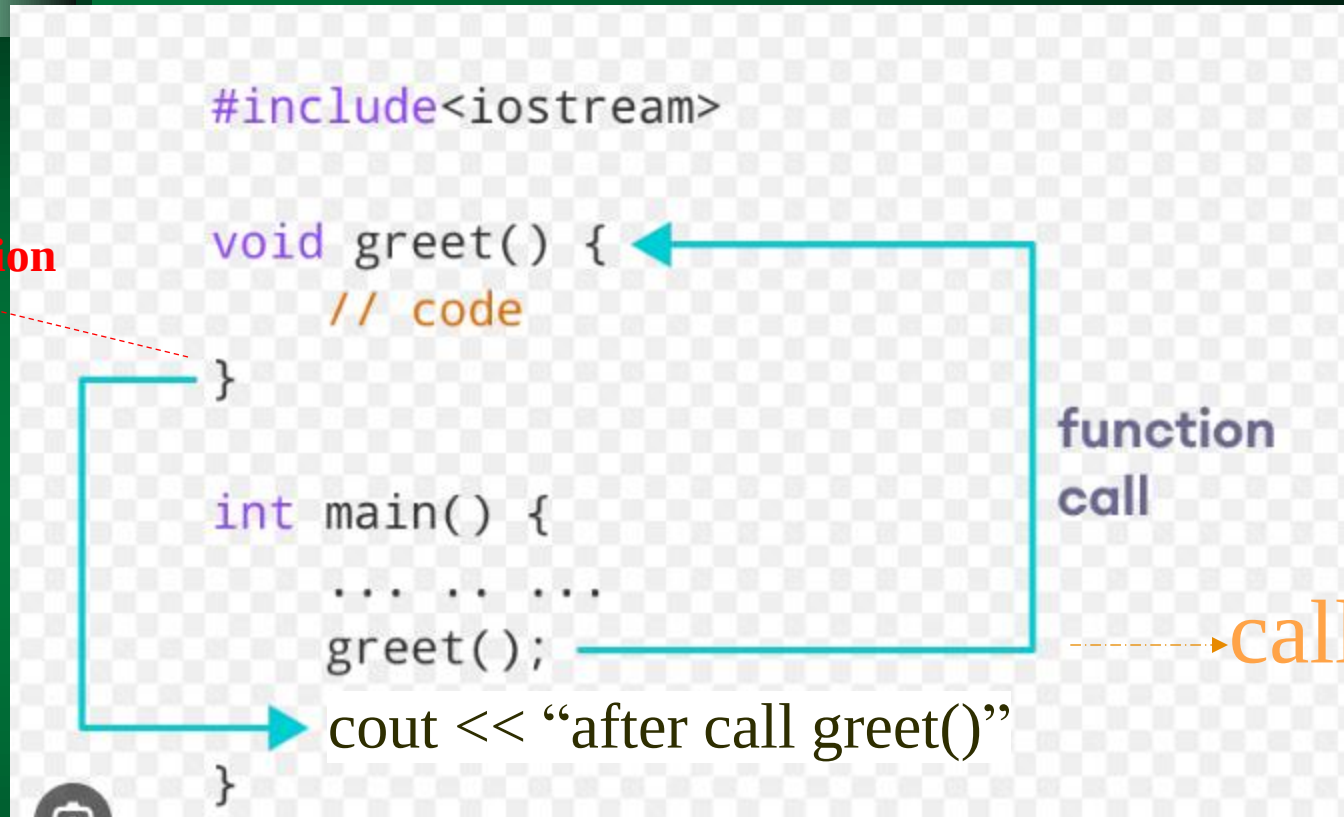
Diagram illustrating the organization of a C/C++ program with annotations:

- Function Parameters:** Points to the parameters `int a, int b` in the function prototype.
- Function Prototype:** Points to the line `int addNumbers(int a, int b);`.
- Main Function:** Points to the `int main()` line.
- Function Call Statement:** Points to the line `sum = addNumbers(n1, n2);`.
- Function Declaration:** Points to the line `int addNumbers(int a, int b)` in the function definition.
- Function Arguments:** Points to the parameters `int a, int b` in the function definition.

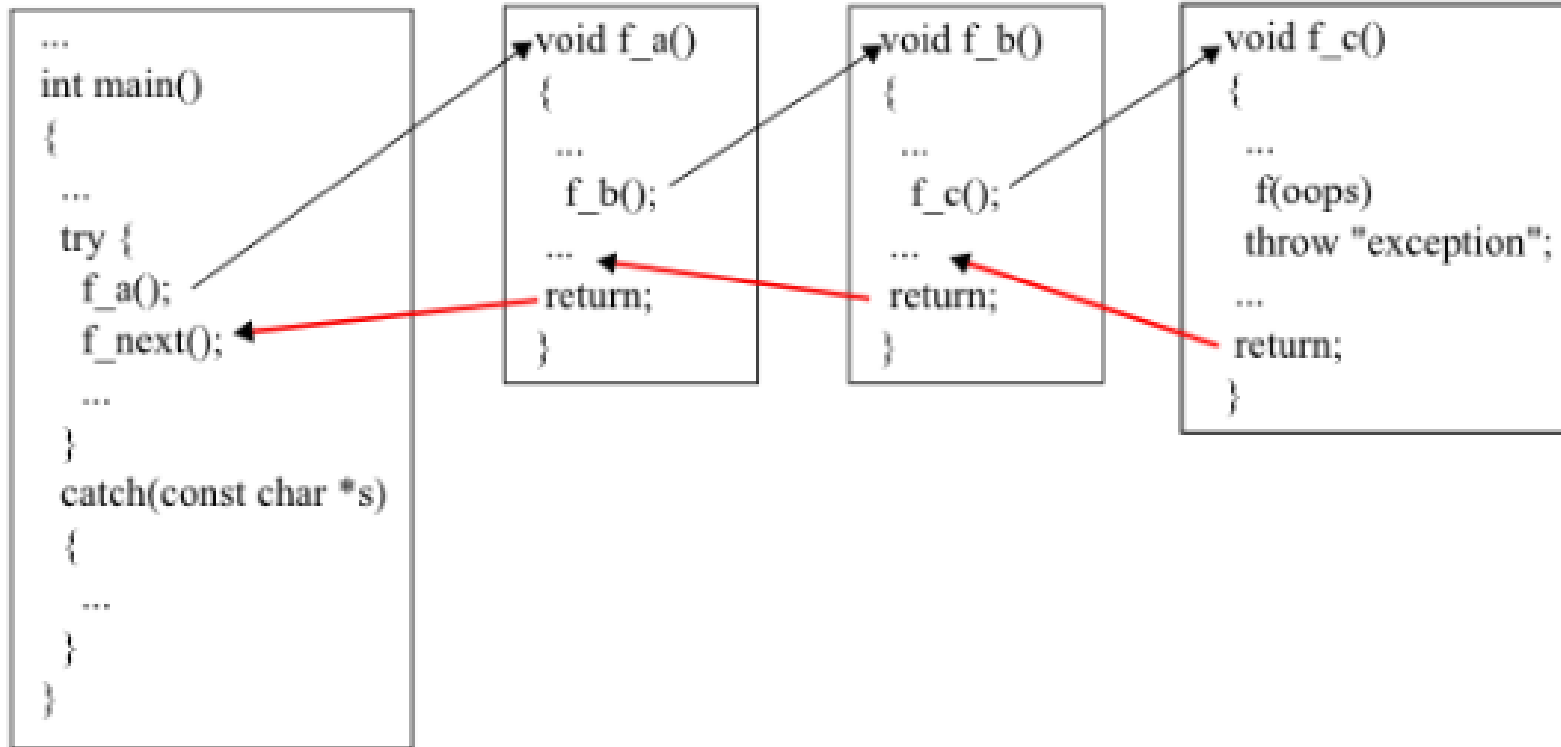
A large red curved arrow on the left side indicates the flow from the function call statement in `main` to the function definition.

Execution flows (1)

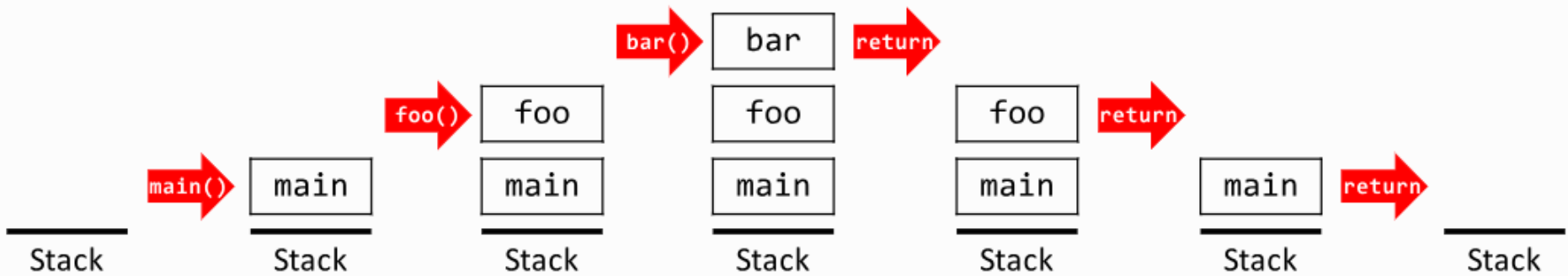
End fuction



Execution flows (2)



Execution flows (3)



Operating System

call

```
void bar() {  
}  
  
void foo() {  
    bar();  
}  
  
int main() {  
    foo();  
}
```

Execution flows : exiting of function in “C/C++ with return statement

```
#include<iostream>
```

```
void greet() {
```

```
// code
```

```
}
```

```
int main() {
```

```
... ..
```

```
greet();
```

```
... ..
```

```
}
```

terminating function with “}”

function call

terminating function with “return” instruction

```
#include<stdio.h>
```

```
int sum(int a, int b)
```

```
{
```

```
return a+b;
```

```
a = a-b;
```

```
}
```

```
int main()
```

```
{
```

```
int ans;
```

```
ans = sum(10,5);
```

```
printf("10 + 5 = %d",ans);
```

```
return 0;
```

Called function

Calling function

Execution will start from here after return



Characteristic of function

1. No return value function (void function)
(procedure subroutine/ void function)
2. Return value function
(function subroutine)



Function Syntax in c/c++

```
return_type function_name ( parameter 1, parameter 2, . . . . )  
    {  
  
        //function body  
  
    }
```

```
// Function to print program title to screen.  
void print_title ( )  
{  
    cout << "Tennis Tournament Scheduler Program\n";  
    cout << "By Jennifer Baker\n";  
}
```

```
12 // Function to print program title to screen.  
13 int get_value_ten ( )  
14 {  
15     return 10;  
16 }  
17
```

c/c++ no return value function (void function)

Variables declare in function body

```
void fname (formal-parameter-declaration-list)
{
    local variable declarations
    executable statements
    return ;
}
```

```
...
int main()
{
    ...
    try {
        f_a();
        f_next();
        ...
    }
    catch(const char *s)
    {
        ...
    }
}
```

```
void f_a()
{
    ...
    f_b();
    ...
    return;
}
```

```
void f_b()
{
    ...
    f_c();
    ...
    return;
}
```

```
void f_c()
{
    ...
    f(oops)
    throw "exception";
    ...
    -----
}
```

```
#include<iostream.h>
```

```
void print_title ( ); // prototype for print_title function
```

```
int main ( )
```

```
{
```

```
    print_title( );    // call to print_title
```

```
    return 0;
```

```
} // end of main function
```

1

5

```
// Function to print program title to screen.
```

```
void print_title ( )
```

```
{
```

```
    cout << "Tennis Tournament Scheduler Program\n";
```

```
    cout << "By Jennifer Baker\n";
```

```
}
```

2

3

4



```
C:\My Documents\c\Release>test  
Tennis Tournament Scheduler Program  
By Jennifer Baker
```

Functions and Program Flow

```
#include<iostream.h>
void print_title( ); // prototype for print_title function
void print_goodbye( ); // prototype for
                        // print_goodbye function
int main()
{
    print_title( ); // call to print_title ← 1
    // insert the rest of the program here
    print_goodbye( ); ← 4
    return 0;
} // end of main function ← 7
```

The diagram illustrates the execution flow of the provided C++ code. Three blue-outlined boxes containing the numbers 1, 4, and 7 are positioned to the right of the code. Arrows point from these boxes to specific lines of code: box 1 points to the call to `print_title( );`, box 4 points to the call to `print_goodbye( );`, and box 7 points to the closing brace of the `main` function.

// Function to print program title to screen.

void print_title()

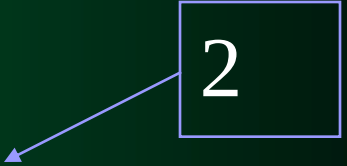
{

cout << "Tennis Tournament Scheduler Program\n";

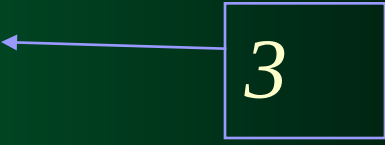
cout << "By Jennifer Baker\n";

}

2



3



// Function to print closing message to screen.

void print_goodbye()

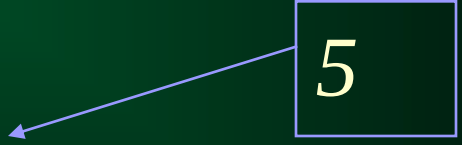
{

cout << "Thank you for using ";

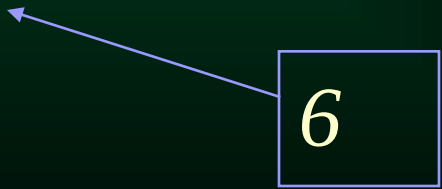
cout << "the Tennis Tournament Scheduler.\n";

}

5



6





Tennis Tournament Scheduler Program

By Jennifer Baker

Thank you for using the Tennis Tournament Scheduler.

c/c++ return value function grammar

```
1  #include<iostream>
2  using namespace std;
3  int get_value_ten ( );    // prototype for print
4  int main ( )
5  {
6      int x;
7
8      x = get_value_ten( );    // call to print
9      cout << x;
10     get_value_ten( );    // call to print_t
11     return 0;
12 } // end of main function
13
14 // Function to print program title to screen.
15 int get_value_ten ( )
16 {
17     return 10;
18 }
19
```

```
ftype fname (formal-parameter-declaration-list)
{
    local variable declarations
    executable statements
    return value;
}
```

1. Ftype : int,float,char, Structure
2. Ftype : array (mention later).

□ Using with “return value” statements

main.cpp

```

1  #include<iostream>
2  using namespace std;
3
4  // Function to print program title to screen.
5  int get_value_ten ( )
6  {
7      return 10;
8  }
9
10 int get_value_two()
11 {
12     return 2;
13 }
14
15 int main ( )
16 {
17     int z;
18
19     z = get_value_ten( ) + get_value_two( );
20     cout << z << endl;
21
22
23     return 0;
24 } // end of main function
25

```

Assign & expression

1. Minus sign used to change sign (-)
2. Multiplication and division (*, /)
3. Addition and subtraction (+, -)
4. assign (=)

ลำดับความสำคัญ
↑ สูง
↓ ต่ำ

ลำดับความสำคัญ →

$z = 1 + 2 * 3$



Var1	Var2	Result
int	int	int
float	float	float
int	float	float

int z;

$z = (1 + 2.1) * 3$



main.cpp

```

5  int get_value_ten ( )
6  {
7      return 10;
8      cout << "get_value_ten ( )" << 10;
9  }
10
11 int get_value_two()
12 {
13     return 2;
14 }
15
16 int main ( )
17 {
18     int z;
19
20     z = get_value_ten( ) + get_value_two( );
21     cout << z << endl;
22
23
24     return 0;
25 } // end of main function
26
27
28

```

Assign & expression

- 1.Minus sign used to change sign (-)
- 2.Multiplication and division (* , /)
- 3.Addition and subtraction (+ , -)
4. assign (=)

ลำดับความสำคัญ

ลำดับความสำคัญ
↑ สูง
↓ ต่ำ

$z = 1 + 2 * 3$



Var1	Var2	Result
int	int	int
float	float	float
int	float	float

int z ;

$z = (1 + 2.1) * 3$



main.cpp

```

1  #include<iostream>
2  using namespace std;
3
4  // Function to print program title to screen.
5  int get_value_ten ( )
6  {
7      return 10;
8  }
9
10 int get_value_two()
11 {
12     return 2;
13 }
14
15 int main ( )
16 {
17     int z;
18
19     z = get_value_ten( ) + get_value_two( );
20     cout << z << endl;
21
22
23     return 0;
24 } // end of main function
25

```

Assign & expression

1. Minus sign used to change sign (-)
2. Multiplication and division (*, /)
3. Addition and subtraction (+, -)
4. assign (=)

ลำดับความสำคัญ
↑ สูง
↓ ต่ำ

ลำดับความสำคัญ →

$$z = 1 + 2 * 3$$

7

6

7

Var1	Var2	Result
int	int	int
float	float	float
int	float	float

int z;

$$z = (1 + 2.1) * 3$$

9

3.1

9.1

```

1  #include<iostream>
2  using namespace std;
3
4  // Function to print program title to screen.
5  int get_value_ten ( )
6  {
7      return 10;
8  }
9
10 int get_value_two()
11 {
12     return 2;
13 }
14
15 int main ( )
16 {
17     int z;
18
19     z = get_value_ten( ) + get_value_two( ) * 100;
20     cout << z << endl;
21
22
23     return 0;
24 } // end of main function
25

```

Assign & expression

1. Minus sign used to change sign (-)
2. Multiplication and division (*, /)
3. Addition and subtraction (+ , -)
4. assign (=)

ลำดับความสำคัญ
↑ สูง
↓ ต่ำ

ลำดับความสำคัญ →

$$z = 1 + 2 * 3$$

7

6

7

Var1	Var2	Result
int	int	int
float	float	float
int	float	float

int z ;

$$z = (1 + 2.1) * 3$$

9

3.1

9.1



Inside “main” function

□ *The C++ program has at least one function as main().*

```
int main( )
```

```
{
```

*Return type of
function*

```
// body of program
```

```
return 0 ;
```

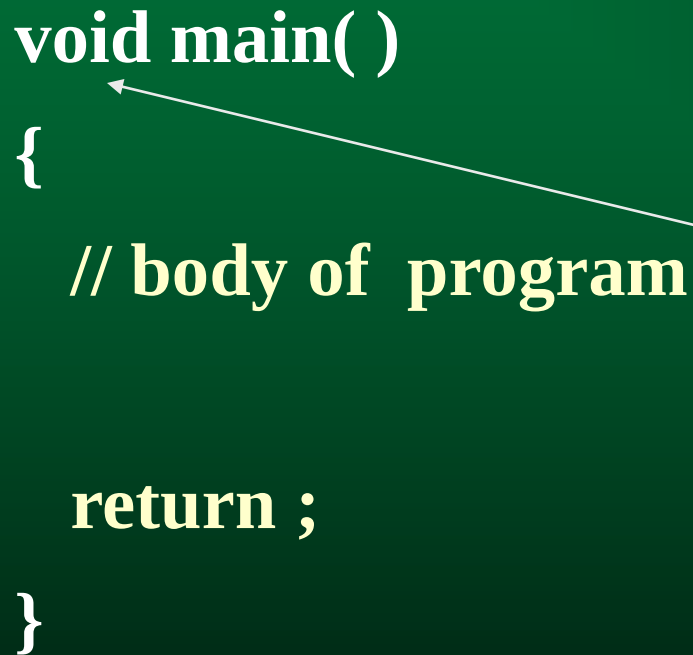
Return value of function

```
}
```

□ *when the program reaches the return 0; statement.
the value zero is returned to the OS.*

Old compiler versions allow “void” on main function

```
void main( )  
{  
    // body of program  
  
    return ;  
}
```



Support “void” on
main function

```
8
9 #include <iostream>
10 using namespace std;
11 int main()
12 {
13     cout<<"Hello World";
14     return 0;
15 }
16
```

Hello World

...Program finished with exit code 0
Press ENTER to exit console.

```
9 #include <iostream>
10 using namespace std;
11 int main()
12 {
13     cout<<"Hello World";
14     //return 0;
15 }
16
```

Hello World

...Program finished with exit code 0
Press ENTER to exit console.

```
9 #include <iostream>
10 using namespace std;
11 int main()
12 {
13     cout<<"Hello World";
14     return 1;
15 }
16
```

Hello World

...Program finished with exit code 1
Press ENTER to exit console.

```
9 #include <iostream>
10 using namespace std;
11 int main()
12 {
13     cout<<"Hello World";
14     return 2;
15 }
16
```

Hello World

...Program finished with exit code 2
Press ENTER to exit console.

□ The value to be return is specified using the *return* statement and the function could return only one value.

```
char getMenuchoice( );           //prototype
char getMenuchoice( )
{
    char select;
    cout << " Menu" << endl;
    cout << "1. Add item to Cart " << endl;
    cout << "2. Sell product " << endl;
    cout << "3. Check stock " << endl;
    cout << "4. End program" << endl;
    cout << "Please select 1 / 2 / 3/ 4 =? ";
    cin >> select;
    return select;
}

int main ( ) {
    char selectMenu ;
    selectMenu = getMenuchoice( );
}
```

```
struct product {  
    char id[10];  
    char name[30];  
    float cost;  
    int instock;  
};
```

return structure

```
struct product getNewProduct() {  
    struct product p;  
    cout << "New product: " << endl;  
    cout << "id =? "; cin >> p.id;  
    cout << "name=? "; cin >> p.name;  
    cout << "cost=? "; cin >> p.cost;  
    cout << "instock=? "; cin >> p.instock;  
    return p;  
}  
int main() {  
    struct product newProduct = getNewProduct();  
  
}
```

Python allows a function return multiple values

main.py

```
1 def operate(a, b):
2     sum = a + b
3     diff = a - b
4     mul = a * b
5     div = a / b
6     return sum, diff, mul, div
7
8 n1 = 5
9 n2 = 10
10 sum, diff, mul, div = operate(n1, n2)
11 print(
12     f"The sum is {sum}\n"
13     f"The difference is {diff}\n"
14     f"The multiplication gives {mul}\n"
15     f"The division gives {div}\n"
16 )
```

Shell

```
The sum is 15
The difference is -5
The multiplication gives 50
The division gives 0.5
```

<https://www.programiz.com/python-programming/online-compiler/>



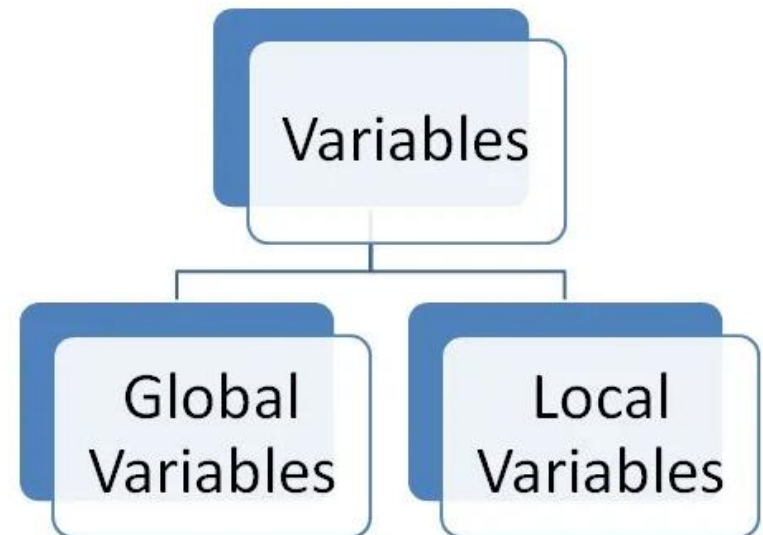
Scope of variable (Global & Local variable)

Scope determines the accessibility (visibility) of variables. c/c++ supports the following scope :

Local Block scope

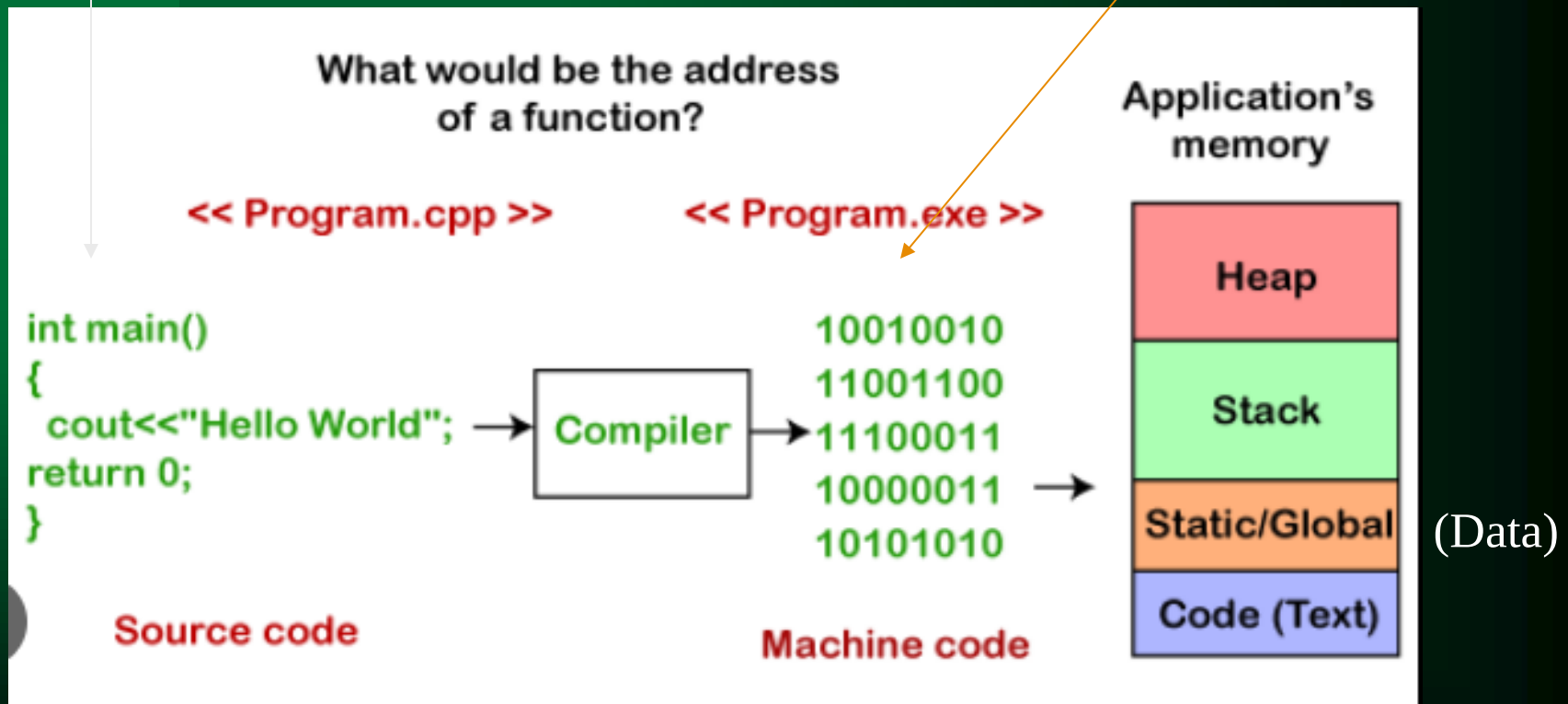
Local Function scope

Global Scope



Recap:

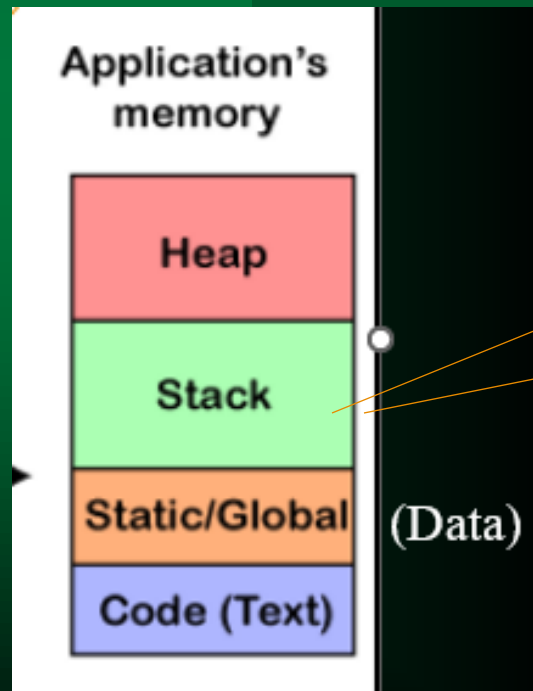
Compiler is a tool that translate a source code to machine code



Global & Local Block & Local scope Property



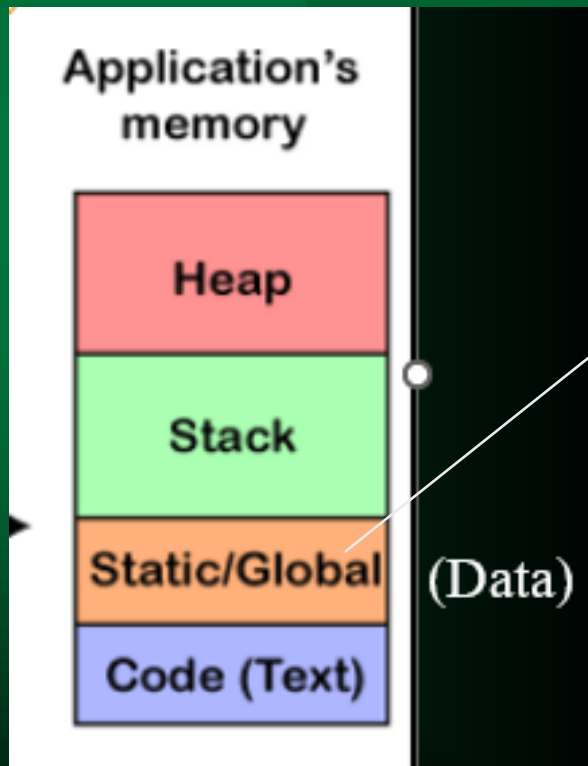
- ❑ Memories of local variables declared within a function are allocated on stack when the function or the block is starting to execute every times.
- ❑ The allocated memories (on stack) are free , when the function or block is existed(terminate)



```
main.cpp
1  #include <iostream>
2  using namespace std;
3  #include <stdio.h>
4  int g = 100; //global variable
5  int main()
6  {
7      int my_num = 7; //local function variable
8      {
9          int new_num = 10; //local block variable
10         int my_num = 17;
11         printf("1. new_num is %d\n",new_num);
12         printf("2.my_num is %d\n",my_num);
13     }
14     printf("3.my_num is %d\n",my_num);
15     {
16         int new_num;
17         my_num = 7*2;
18         printf("5. new_num is %d\n",new_num);
19         printf("6.my_num is %d\n",my_num);
20     }
```

Global & Local Block & Local scope Property

- ❑ Memory of global variables are allocated on data(global) segment when the program starts.
- ❑ The memories are freed when the program terminates.



main.cpp

```
1  #include <iostream>
2  using namespace std;
3  #include <stdio.h>
4  int g = 100; //global variable
5  int main()
6  {
7      int my_num = 7; //local function variable
8      {
9          int new_num = 10; //local block variable
10         int my_num = 17;
11         printf("1. new_num is %d\n", new_num);
12         printf("2. my_num is %d\n", my_num);
13     }
14     printf("3. my_num is %d\n", my_num);
15     {
16         int new_num;
17         my_num = 7*2;
18         printf("5. new_num is %d\n", new_num);
19         printf("6. my_num is %d\n", my_num);
20     }
```

Block scope vs local scope

main.cpp

```
2 using namespace std;
3 #include <stdio.h>
4 int main()
5 {
6     int my_num = 7;
7     {
8         int new_num = 10;
9         int my_num = 17;
10        printf("1. new_num is %d\n",new_num);
11        printf("2.my_num is %d\n",my_num);
12    }
13    printf("3.my_num is %d\n",my_num);
14    {
15        int new_num;
16        my_num = 7*2;
17        printf("5. new_num is %d\n",new_num);
18        printf("6.my_num is %d\n",my_num);
19    }
20    printf("7.my_num is %d\n",my_num);
21    return 0;
22 }
```

1. new_num is 10
2.my_num is 17
3.my_num is 7
5. new_num is 0
6.my_num is 14
7.my_num is 14

Block scope vs local scope

main.cpp

```
2 using namespace std;
3 #include <stdio.h>
4 int main()
5 {
6     int my_num = 7;
7     {
8         int new_num = 10;
9         int my_num = 17;
10        printf("1. new_num is %d\n",new_num);
11        printf("2.my_num is %d\n",my_num);
12    }
13    printf("3.my_num is %d\n",my_num);
14    {
15        // int new_num;
16        my_num = 7*2;
17        printf("5. new_num is %d\n",new_num);
18        printf("6.my_num is %d\n",my_num);
19    }
20    printf("7.my_num is %d\n",my_num);
21    return 0;
22 }
```

input

Compilation failed due to following error(s).

main.cpp: In function 'int main()':

main.cpp:17:37: error: 'new_num' was not declared in this scope

```
17 |         printf("5. new_num is %d\n",new_num);
    |                                     ^~~~~~
```

Block scope vs local scope

```
1 #include <iostream>
2 using namespace std;
3 #include <stdio.h>
4 int main()
5 {
6     int my_num = 7;
7     {
8         int new_num = 10;
9         int my_num = 17;
10        printf("1. new_num is %d\n",new_num);
11        printf("2.my_num is %d\n",my_num);
12    }
13    printf("3.my_num is %d\n",my_num);
14    {
15        int new_num;
16        my_num = 7*2;
17        printf("5. new_num is %d\n",new_num);
18        printf("6.my_num is %d\n",my_num);
19    }
20    printf("7.my_num is %d\n",my_num);
21    return 0;
22 }
```

1. new_num is 10
2.my_num is 17
3.my_num is 7
5. new_num is 0
6.my_num is 14
7.my_num is 14

main.cpp

```
1 //using namespace std;
2 #include <stdio.h>
3 int main()
4 {
5     int my_num = 7;
6     {
7         int new_num = 10;
8         int my_num = 17;
9         printf("1. new_num is %d\n",new_num);
10        printf("2.my_num is %d\n",my_num);
11    }
12    printf("3.my_num is %d\n",my_num);
13    {
14        //int new_num = 10;
15        my_num = 7*2;
16        printf("5. new_num is %d\n",new_num);
17        printf("6.my_num is %d\n",my_num);
18    }
19    printf("7.my_num is %d\n",my_num);
20    return 0;
21 }
22
23
24
```

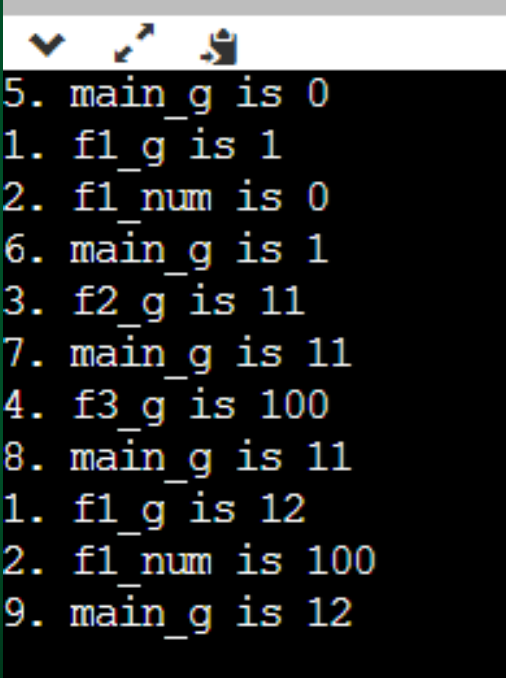
input

Compilation failed due to following error(s).

```
main.cpp: In function 'int main()':
main.cpp:17:37: error: 'new_num' was not declared in this scope
17 |         printf("5. new_num is %d\n",new_num);
   |                                     ^~~~~~
```

Global vs local scope

```
main.cpp (Ctrl+M)
1  #include <iostream>
2  using namespace std;
3  #include <stdio.h>
4  int g = 0 ;
5  void f1() {
6      int f1_num;
7      g++;
8      printf("1. f1_g is %d\n",g);
9      printf("2. f1_num is %d\n",f1_num);
10     f1_num++;
11 }
12
13 void f2() {
14     g += 10;
15     printf("3. f2_g is %d\n",g);
16 }
17
18 void f3() {
19     int g = 100;
20     printf("4. f3_g is %d\n",g);
21 }
22 int main()
23 {
24     printf("5. main_g is %d\n",g);
25     f1();
26     printf("6. main_g is %d\n",g);
27     f2();
28     printf("7. main_g is %d\n",g);
29     f3();
30     printf("8. main_g is %d\n",g);
31     f1();
32     printf("9. main_g is %d\n",g);
33 }
34
```



```
5. main_g is 0
1. f1_g is 1
2. f1_num is 0
6. main_g is 1
3. f2_g is 11
7. main_g is 11
4. f3_g is 100
8. main_g is 11
1. f1_g is 12
2. f1_num is 100
9. main_g is 12
```

Global vs local scope

main.cpp

```
1  #include <iostream>
2  using namespace std;
3  #include <stdio.h>
4  int g = 0 ; //global
5  void f1() {
6      int f1_num;
7      g++;
8      printf("1. f1_g is %d\n",g);
9      printf("2. f1_num is %d\n",f1_num);
10     f1_num++;
11 }
12
13 void f2() {
14     f1_num++;    //local from f1
15     g += 10;
16     printf("3. f2_g is %d\n",g);
17 }
18
19 void f3() {
20     int g = 100;
21     printf("4. f3_g is %d\n",g);
22 }
23 int main()
24 {
25     printf("5. main_g is %d\n",g);
26     f1();
27     printf("6. main_g is %d\n",g);
28     f2();
29     printf("7. main_g is %d\n",g);
30     f3();
31     printf("8. main_g is %d\n",g);
32     f1();
33     printf("9. main_g is %d\n",g);
34 }
35
```

Compilation failed due to following error(s).

main.cpp: In function 'void f2()':

main.cpp:14:5: error: 'f1_num' was not declared in this scope

```
14 |     f1_num++;    //local from f1
    |     ^~~~~~
```

```

4 using namespace std;
5 int g = 10 , i = 3;    // global variable
6 void myfunction();
7 int main()
8 {
9     int j , k;
10    j = 2;
11    k = i + j;
12    cout << "\n1.main k :" << k ;
13    cout << "\n1.main i :" << i ;
14    myfunction( ); // call to myfunction
15    cout << "\n2.main k :" << k ;
16    cout << "\n2.main j :" << j ;
17    cout << "\n2.main i :" << i ;
18    g++;
19    myfunction( ); // call to myfunction
20    cout << "\n3.main k :" << k ;
21    cout << "\n3.main j :" << j ;
22    cout << "\n3.main g :" << g ;
23    cout << "\n4.main i :" << i ;
24    return 0;
25 }
26 void myfunction()
27 {
28     int k;    // local variable
29     int g = 40;
30     k = i++; g = k ;
31     cout << "\nmyfunction k :" << k ;
32     cout << "\nmyfunction g :" << g ;
33 }
34

```

k = ?

j = ?

g = ?

i = ?

```
#include<iostream.h>
void myfunction();
int main()
{
    int j,k,i; // variables local to the main function
    j = 2;
    cin >> i ;
    k = i + j;
    cout << "j = " << j << " and k = " << k << '\n';
    cout << "i = " << i << " before the call to myfunction.\n";
    myfunction( ); // call to myfunction
    cout << "i = " << i << " after the call to myfunction.\n";
    return 0;
}

void myfunction()
{
    int x=5;
    x = ++i;
    cout << "i = " << x << '\n';
}
```

i = ?

SCOPE1.CPP

```
int j,k,i;    // variables local to the main function
j = 2;
cin >> i ;
k = i + j;
cout << "j = " << j << " and k = " << k << '\n';
cout << "i = " << i << " before the call to myfunction.\n";
myfunction(); // call to myfunction
cout << "i = " << i << " after the call to myfunction.\n";
return 0;
}
```

```
void myfunction()
```

```
{
    int x=5;
    x = ++i;
    cout << "i = " << x << '\n';
}
```

Error :using i out of scope

18:11

Message

Compiling SCOPE1.CPP:

Error SCOPE1.CPP 18: Undefined symbol 'i'

```

#include<iostream.h>
int g = 10 , i = 3;  // global variable
void myfunction();
int main()
{
    int j , k;  // variables local to the main function
    j = 2;    k = i + j;
    cout << "before g = " << g << " \n";
    cout << "j = " << j << " and k = " << k << '\n';
    cout << "i = " << i
        << " before the call to  myfunction.\n";
    myfunction( ); // call to myfunction
    cout << "i = " << i
        << " after the call to myfunction.\n";
    cout << "after g = " << g ;
    return 0;
}

```

```

void myfunction()
{
    int l;    // local variable
    int g = 40;
    l = ++i;
    cout << "l = " << l << '\n';
    cout << "The variable l is lost
        as soon as  myfunction exits.\n";

    cout << "g (myfunction) = "
        << g << " \n";
    g = 80;
}

```

ผลของการ execute โปรแกรม คือ ?

```
#include<iostream.h>
```

```
int i = 3; // global variable
```

```
void myfunction();
```

```
int main()
```

```
{
```

```
    int j , k; // variables local to the main function
```

```
    j = 2;
```

```
    k = i + j;
```

```
    cout << "j = " << j << " and k = " << k << '\n';
```

```
    cout << "i = " << i << " before the call to myfunction.\n";
```

```
    myfunction( ); // call to myfunction
```

```
    cout << "i = " << i << " after the call to myfunction.\n";
```

```
    return 0;
```

```
}
```

ผลของการ execute โปรแกรม คือ ?

```
void myfunction()
```

```
{
```

```
    int l; // local variable
```

```
    l = ++i; // the variable i is accessible because i is global
```

```
            // the variable i is changed globally
```

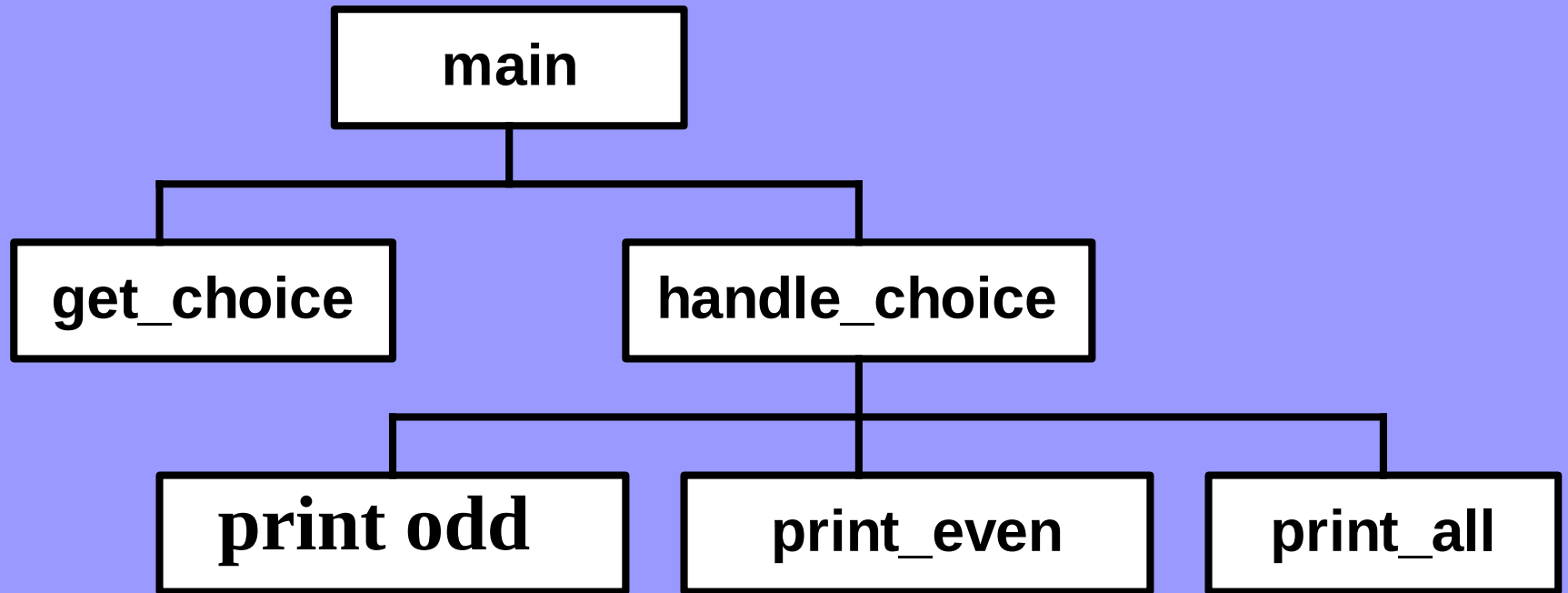
```
    cout << "l = " << l << '\n';
```

```
    cout << "The variable l
```

```
            is lost as soon as myfunction exits.\n";
```

```
}
```

Home work



- rewrite a **example1_1.cpp** program by dividing activities to correspond with about diagram.

```

1 #include <iostream>
2 using namespace std;
3
4 int getchoice() {
5     int choice; // variable for user input
6     do // loop until a valid choice is entered
7     {
8         cout << "Which series do you wish to display?\n";
9         cout << "1 - Odd numbers from 1 to 30\n";
10        cout << "2 - Even numbers from 1 to 30\n";
11        cout << "3 - All numbers from 1 to 30\n";
12        cin >> choice; // get choice from user
13        if ((choice < 1) || (choice > 3))
14        { // if invalid entry, give message
15            cout << "Choice must be 1, 2, or 3\n";
16        }
17    } while ((choice < 1) || (choice > 3));
18    return choice;
19 }
20
21 void printOdd() {
22     for (int i = 1; i <= 30; i = i + 2)
23         cout << i << ' ';
24     cout << endl;
25 }
26
27 void printEvent() {
28     for (int i = 2; i <= 30; i = i + 2)
29         cout << i << ' ';
30     cout << endl;
31 }
32

```

```

33 void printAll() {
34     for (int i = 2; i <= 30; i = i + 2)
35         cout << i << ' ';
36     cout << endl;
37 }
38
39 int main()
40 {
41
42     int choice = getchoice();
43     switch (choice)
44     {
45     case 1:
46         printOdd();
47         break;
48     case 2:
49         printEvent() ;
50         break;
51     case 3:
52         printAll( );
53         break;
54     }
55     return 0;
56 }
57

```

7.2 parameter passing

Passing/Getting Data to and from function

- There are two ways to pass data to function in C++
 - **Passing by value** : send data to process on a function
 - **Passing by reference** : retrieve/update data from a function

7.2 Passing Data

- Memory of parameters on function are allocated on stack.
- They are allocated during a function is executing ,and are deallocated after the function is ended like local parameters.
- The initial values of parameters is from argument variables.

```
void pass_val ( int x , double &y ) ;
```

```
int main()
```

```
{ double y = 18.0 ;
```

```
  int x = 3 ;
```

```
  pass_val ( x , y )
```

```
  cout << x << y << endl ;
```

```
}
```

```
void pass_val ( int l , double &y )
```

```
{
```

```
  l = l*2; y = y+2;
```

```
  cout << l << y << endl ;
```

```
}
```

arguments

-x = passed by values
-y = pass by reference

parameters

Application's memory

Heap

Stack

Static/Global

Code (Text)

(Data)

Grammar : Function Body

```
2  #include <iostream>
3
4  using namespace std;
5
6  void pass_val ( int x , double &y ) ;
7  int main()
8  {
9      double y = 18.0 ;
10     int x = 3 ;
11     pass_val ( x , y );
12     cout << "\nx = " << x;
13     cout << "\ny = " << y ;
14 }
15 void pass_val ( int l , double &y)
16 {
17     l = l*2;  y = y+2;
18     cout << "\nl = " << l;
19     cout << "\ny = " << y ;
20 }
21
```

arguments

-x = passed by values
-y = pass by reference

parameters



```
5 void pass_val ( int x , double &y ) ;
6 int main()
7 {
8     double y = 18.0 ;
9     int x = 3 ;
10    int i=100;
11    double j=91.1;
12    cout << "\nmain before -> x = " << x;
13    cout << "\nmain before -> y = " << y ;
14
15    pass_val ( x , y );
16    cout << "\nmain -> x = " << x;
17    cout << "\nmain -> y = " << y ;
18
19    cout << "\nmain before -> i = " << i;
20    cout << "\nmain before -> j = " << j ;
21
22    pass_val ( i , j );
23    cout << "\nmain -> i = " << i;
24    cout << "\nmain -> j = " << j ;
25
26 }
27 void pass_val ( int l , double &a)
28 {
29     cout << "\npass_val before -> l = " << l;
30     cout << "\npass_val before -> a = " << a ;
31
32     l = l*2;  a = a+2;
33     cout << "\npass_val -> l = " << l;
34     cout << "\npass_val -> a = " << a ;
35 }
36
```

7.2 Passing Data *Pass by value*

- The objective of design function with “pass by value” is the sending data from the caller to process in that function routine.
- The mechanism is the copying values of argument variables to be the initial value of parameter variables, so updating parameter variables in the calling function the argument value remain the same.

Parameter : formal parameters

Argument : actual parameters

The diagram shows two code snippets separated by a dashed line. The top snippet is for `MyProgram` and the bottom is for `ToolBox`. In `MyProgram`, the `main` method has local variables `x` and `y` assigned values, and a call to `ToolBox.min(x, y)`. In `ToolBox`, the `min` method has formal parameters `a` and `b`. Arrows labeled "Actual parameters" point from `x` and `y` in `MyProgram` to the `min` call. Another set of arrows labeled "Formal parameters" points from the `min` call in `MyProgram` to the parameters `a` and `b` in the `min` method of `ToolBox`.

```
public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x, y );
        ....
    }
}

public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}
```

ตย1. กลไกการ pass by value

```
public class MyProgram
{
    public static void main ( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x , y );

        ....

    }
}
```

```
public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}
```

RAM memory

main method

min method



```

public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x , y );
        ....
    }
}

```

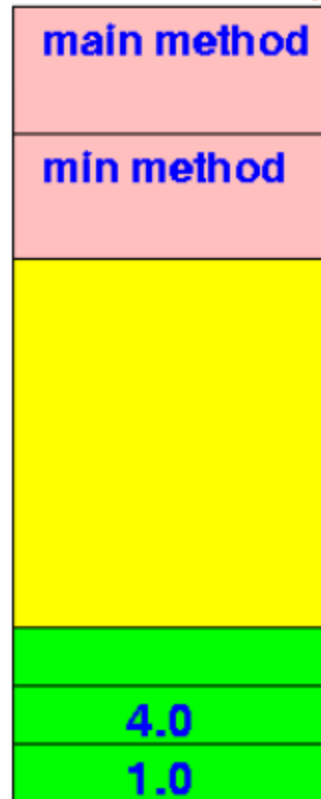
```

public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}

```

main creates its local variables

RAM memory



```

public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        → r = ToolBox.min( x, y );
        ....
    }
}

```

```

public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}

```

Pass-by-value:
1. create parameter variables

RAM memory



```

public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        ➔ r = ToolBox.min( x, y );
        ....
    }
}

```

```

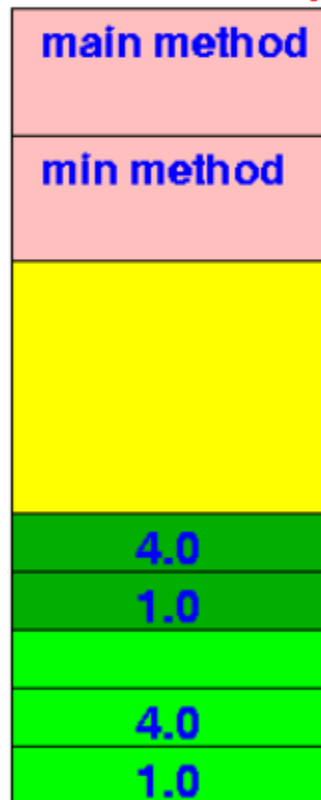
public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}

```

Pass-by-value:

2. copy value of actual parameter to formal parameter

RAM memory



```

public class MyProgram
{
    public static void main( String[] args )
    {

```

```

        double x = 1.0, y = 4.0;
        double r;

```

```

        r = ToolBox.min( x, y );

```

```

        ....
    }
}

```

```

public class ToolBox
{
    public static double min ( double a, double b )
    {

```

```

        double m = 0;

```

```

        if ( a < b )
        {

```

```

            m = a;
        }

```

```

        else
        {

```

```

            m = b;
        }

```

```

        return(m);
    }
}

```

Method invocation mechanism:
Jump to method

RAM memory

main method

min method

return address

b	4.0
a	1.0
r	
y	4.0
x	1.0

```

public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x, y );
        ....
    }
}

```

Method invocation mechanism:
Jump to method

RAM memory

main method	
min method	
m	0
return address	
b	4.0
a	1.0
r	
y	4.0
x	1.0

```

public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}

```

create local variable

```

public class MyProgram
{
    public static void main (String[] args)
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x, y );
        ....
    }
}

```

Method access information by:
*Obtain information from
 parameter variable*

```

public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}

```

RAM memory

	main method
	min method
m	0
	<i>return address</i>
b	4.0
a	1.0
r	
y	4.0
x	1.0

```

public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x, y );
        ....
    }
}

```

*Method access information by:
Obtain information from
parameter variable*

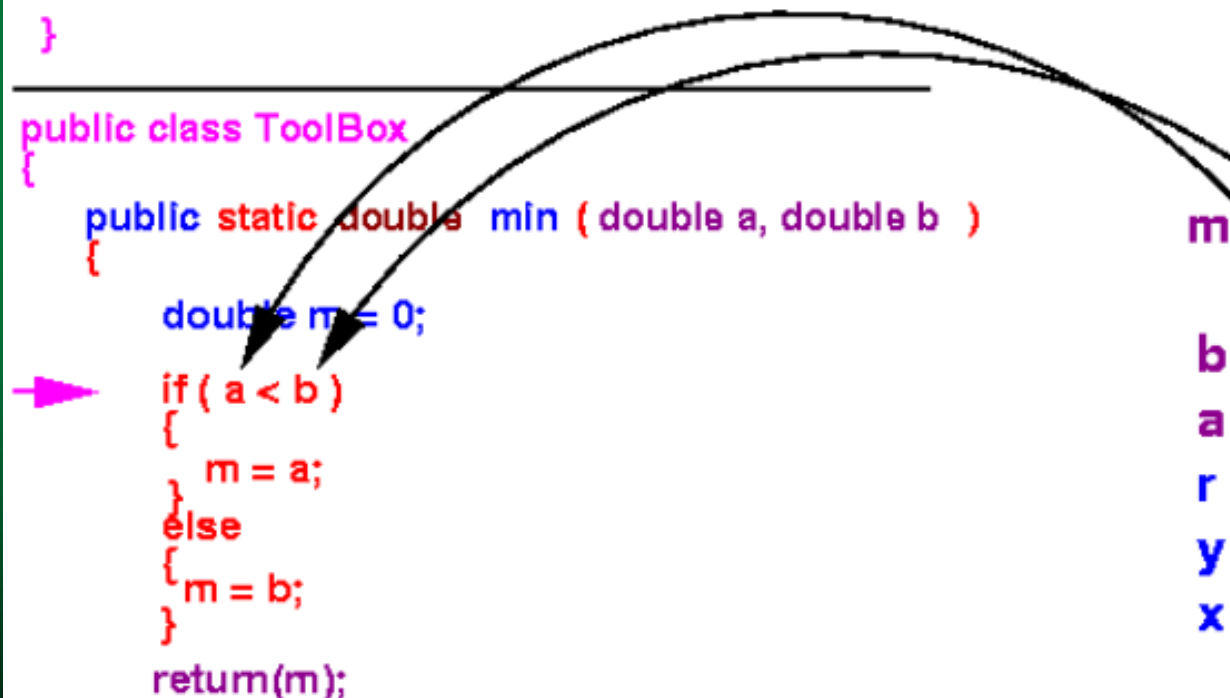
RAM memory

```

public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}

```

main method	
min method	
m	0
<i>return address</i>	
b	4.0
a	1.0
r	
y	4.0
x	1.0



Ex2 :

```
public class MyProgram
{
    public static void main (String[] args)
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.fun( x , y );

        System.out.println(x);
        System.out.println(y);
        System.out.println(r);
    }
}
```

```
public class ToolBox
{
    public static double fun ( double a, double b )
    {
        double m = 0;
        a = a + 1;
        b = b + 2;
        m = a + b;
        return(m);
    }
}
```

RAM memory

main method

min method

Update parameter variable !

```

public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.fun( x , y );

        System.out.println(x);
        System.out.println(y);
        System.out.println(r);
    }
}

```

```

public class ToolBox
{
    public static double fun ( double a, double b )
    {
        double m = 0;
        → a = a + 1;
        b = b + 2;
        m = a + b;
        return(m);
    }
}

```

Situation when method fun starts running

RAM memory

	main method
	min method
m	0
	return address
b	4.0
a	1.0
r	
y	4.0
x	1.0

```

public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.fun( x , y );

        System.out.println(x);
        System.out.println(y);
        System.out.println(r);
    }
}

```

```

public class ToolBox
{
    public static double fun ( double a, double b )
    {
        double m = 0;
        a = a + 1;
        b = b + 2;
        m = a + b;
        return(m);
    }
}

```

Assignment statements changes values stored in parameter var

RAM memory

	main method
	min method
m	0
	<i>return address</i>
b	4.0 6.0
a	1.0 2.0
r	
y	4.0
x	1.0

- The *values* in the *actual parameters* (x and y) are *unchanged* !!!

```
public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.fun( x , y );

        System.out.println(x);
        System.out.println(y);
        System.out.println(r);
    }
}
```

```
public class ToolBox
{
    public static double fun ( double a, double b )
    {
        double m = 0;
        a = a + 1;
        b = b + 2;
        m = a + b;
        return(m);
    }
}
```

That's why the statements

```
System.out.println(x);    ---> prints 1.0
System.out.println(y);    ---> prints 4.0
```

Actual parameters not affected !!

RAM memory

	main method
	min method
m	0
	return address
b	4.0 6.0
a	1.0 2.0
r	
y	4.0
x	1.0

Passing by value in C/C++ :

main.cpp

```
1
2 #include <iostream>
3
4 using namespace std;
5
6 void pass_val ( int , double ) ;
7 int main()
8 {
9     double y = 18.0 ;
10    int x = 3 ;
11    pass_val ( x , y );
12    cout << "\nx = " << x;
13    cout << "\ny = " << y;
14 }
15 void pass_val ( int a , double b )
16 {
17     a = a*2; b = b+2;
18     cout << "\na = " << a;
19     cout << "\nb = " << b;
20 }
21
22
```

X=?

Y=?

arguments

18.0

3

parameters

```
#include <iostream.h>
```

```
void print_value( int j);
```

```
int main()
```

```
{
```

```
    int i = 2;
```

```
    cout << "Value before = " << i << endl ;
```

```
    print_value( i );
```

```
    cout << "Value after = " << i << endl ;
```

```
    return 0;
```

```
}
```

```
void print_value( int j )
```

```
{
```

```
    cout << "value passed = " << j << endl ;
```

```
    j = j*2 ;
```

```
    cout << "value at end func = " << j << endl ;
```

```
}
```

```
C:\My Documents\sc107>passval1
```

```
Value before = 2
```

```
value passed = 2
```

```
value at end func = 4
```

```
Value after = 2
```

```
#include <iostream.h>
void print_value( int j);
int main()
{
    int i = 2;
    cout << "Value before = " << i << endl ;
    print_value( i );
    cout << "Value after = " << i << endl ;
    i = 40;
    print_value( i );
    cout << "Value after = " << i << endl ;
    return 0;
}
void print_value( int j )
{
    cout << "value passed = " << j << endl ;
    j = j*2 ;
    cout << "value at end func = " << j << endl ;
}
```

**What are the value of i , j
after main function call
print_value function ?**

main.cpp

```
1 #include <iostream>
2 using namespace std;
3
4 struct employee {
5     char name[100];
6     int age;
7     float salary;
8     char department[50];
9 };
10
11 // This function takes structure variable as parameter
12 void printEmployeeDetails(struct employee emp){
13     cout << "\n*** Employee Details ***\n";
14     cout << "Name : " << emp.name << "\nAge : " << emp.age << "\nSalary : "
15         << emp.salary << "\nDepartment : " << emp.department;
16 }
17 int main(){
18     struct employee manager ;
19
20     printf("Enter Name, Age, Salary and Department of Employee\n");
21     // Assigning data to members of structure variable
22     cin >> manager.name >> manager.age >> manager.salary >> manager.department;
23
24     // Passing structure variable to function
25     printEmployeeDetails(manager);
26 }
```



Enter Name, Age, Salary and Department of Employee

apirak

30

1000.5

ram

*** Employee Details ***

Name : apirak

Age : 30

Salary : 1000.5

Department : ram



7.2 Passing Data

Passing by reference

Parameter : formal parameters
Argument : actual parameters

- The pass by reference is a two ways communication between the caller and its called function.
 - One for sending data from the caller to the function
 - One for receiving data of caller from its function
- However, the mainly objective of the method is getting information from other function.

Pass-by-reference mechanism

ตย2. กลไกการ pass by reference

```
public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x , y );
        ....
    }
}
```

```
public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}
```

main creates its local variables

RAM memory

main method

min method

49997 r

49998 y

49999 x

4.0

1.0

```

public class MyProgram
{
    public static void main (String[] args)
    {
        double x = 1.0, y = 4.0;
        double r;
        → r = ToolBox.min( x, y );
        ....
    }
}

```

Pass-by-reference:
1. create parameter variables

```

public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}

```

RAM memory



```

public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x, y );
        ....
    }
}

```

```

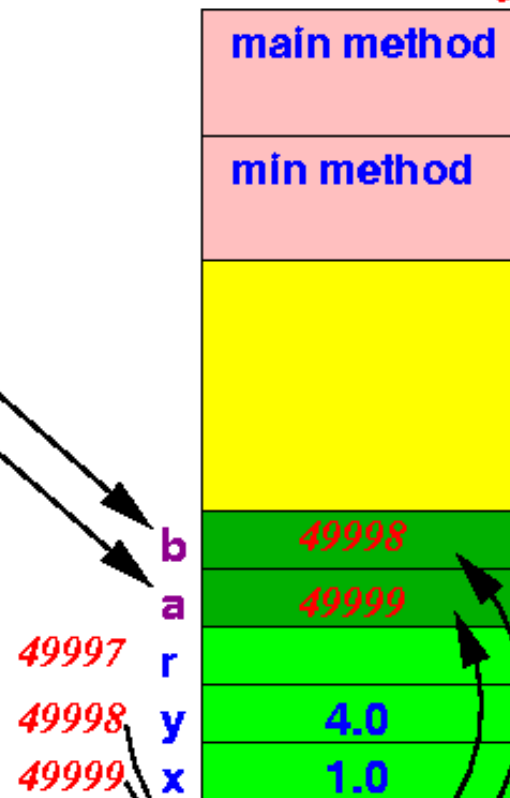
public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}

```

Pass-by-reference:

2. copy reference of actual parameter to formal parameter

RAM memory



```

public class MyProgram
{
    public static void main (String[] args)
    {

```

Method invocation mechanism:
Save return address

```

        double x = 1.0, y = 4.0;
        double r;

```

➡ `r = ToolBox.min( x , y );` ← *return address*

```

        ....
    }
}

```

```

public class ToolBox
{
    public static double min ( double a, double b )
    {
        double m = 0;
        if ( a < b )
        {
            m = a;
        }
        else
        {
            m = b;
        }
        return(m);
    }
}

```

RAM memory

main method	
min method	
return address	
b	49998
a	49999
r	
y	4.0
x	1.0

```

public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x , y );
        ....
    }
}

```

Method invocation mechanism:
Jump to method

RAM memory

main method	
min method	
0	
return address	
b	49998
a	49999
r	
y	4.0
x	1.0

```

public class ToolBox
{

```

```

    public static double min ( double a, double b )
    {

```

```

        double m = 0;

```

```

        if ( a < b )
        {

```

```

            m = a;

```

```

        }
        else
        {

```

```

            m = b;

```

```

        }
        return(m);
    }
}

```

```
public class MyProgram
```

```
{
```

```
    public static void main (String[] args)
```

```
{
```

```
        double x = 1.0, y = 4.0;
```

```
        double r;
```

```
        r = ToolBox.min( x, y ); ← return address
```

```
        ....
```

```
    }
```

```
}
```

```
public class ToolBox
```

```
{
```

```
    public static double min ( double a, double b )
```

```
{
```

```
        double m = 0;
```

```
→    if ( a < b )
```

```
    {
```

```
        m = a;
```

```
    }
```

```
    else
```

```
    {
```

```
        m = b;
```

```
    }
```

```
        return(m);
```

```
    }
```

```
}
```

Method access information by:
1. Locate the actual parameter

RAM memory

main method

min method

0

return address

49998

49999

4.0

1.0

49997

49998

49999

m

b

a

r

y

x

```

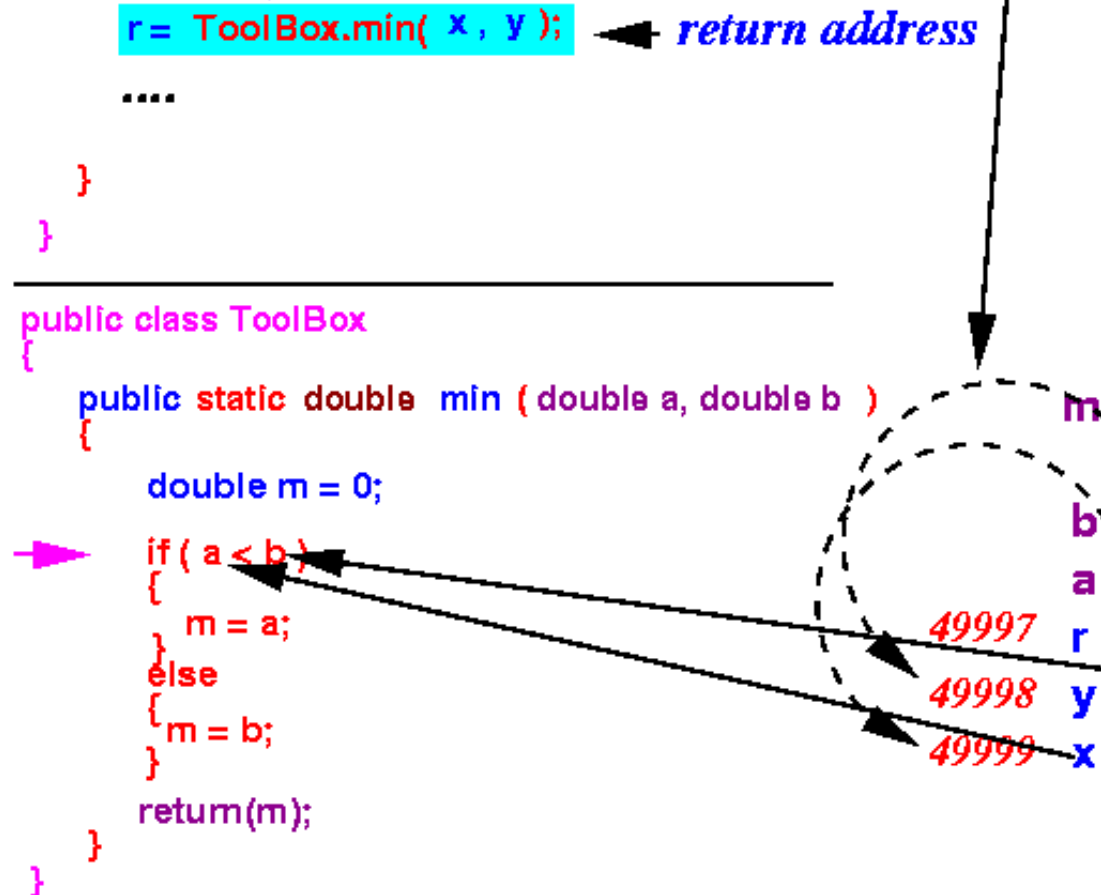
public class MyProgram
{
    public static void main (String[] args)
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.min( x, y );
        ....
    }
}

```

Method access information by:
2. Extract the information from the actual parameters

RAM memory

main method
min method
0
return address
49998
49999
4.0
1.0



Pass-by-reference mechanism update value case

```
public class MyProgram
{
    public static void main( String[] args )
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.fun( x, y );
        System.out.println(x);
        System.out.println(y);
        System.out.println(r);
    }
}
```

```
public class ToolBox
{
    public static double fun ( double a, double b )
    {
        double m = 0;
        → a = a + 1;
        b = b + 2;
        m = a + b;
        return(m);
    }
}
```

Situation when method fun starts running

RAM memory

	main method
	min method
m	0
	<i>return address</i>
b	49998
a	49999
49997 r	
49998 y	4.0
49999 x	1.0

```

public class MyProgram
{
    public static void main (String[] args)
    {
        double x = 1.0, y = 4.0;
        double r;
        r = ToolBox.fun( x , y );
        System.out.println(x);
        System.out.println(y);
        System.out.println(r);
    }
}

```

Assignment statements changes values stored in actual parameter

```

public class ToolBox
{
    public static double fun ( double a, double b )
    {
        double m = 0;
        a = a + 1;
        b = b + 2;
        m = a + b;
        return(m);
    }
}

```

RAM memory

main method	
min method	
0	
return address	
b	49998
a	49999
r	
y	4.0
x	1.0 2.0

1.0

2.0

Example of pass by reference

main.cpp

```
1  #include <stdio.h>
2
3  void swapnum(int &i, int &j) {
4      int temp = i;
5      i = j;
6      j = temp;
7  }
8
9  int main(void) {
10     int a = 10;
11     int b = 20;
12
13     printf("before : A is %d and B is %d\n", a, b);
14     swapnum(a, b);
15     printf("after : A is %d and B is %d\n", a, b);
16     return 0;
17 }
```

before : A is 10 and B is 20
after : A is 20 and B is 10

...Program finished with exit code 0



7.2 Passing Data

Passing by reference

```
void get_values ( float &income , float &expense );
```

```
void get_values ( float &income , float &expense )  
{  
    cout << " Enter income amount " ;  
    cin >> income ;  
    cout << " Enter expense amount " ;  
    cin >> expense ;  
}
```



C/C++ Pass by reference

Parameter : formal parameters
Argument : actual parameters

- ❑ Support all data types using “&” on actual parameter
- ❑ By default ,passing “Array data type” is pass by reference
- ❑ The formal parameters of the caller must be “variable” only.

```
#include <iostream.h>
void get_value( int &j );
int main()    !! Compiler error
{
    int i = 0 ;
    cout << "before call i = " << i << endl ;
    get_value( 10 ) ;
    cout << "after call i = " << i << endl ;
    return 0;
}

void get_value( int &j )
{
    cout << "value passed to func = " << j << endl ;
    cin >> j ;
    cout << "value at end func = " << j << endl ;
}
```

```

7
8
9 #include <iostream>
10 using namespace std;
11 struct myStructure {
12     int x;
13     int y;
14 };
15
16 void swap(myStructure &ms)
17 {
18     int temp = ms.x;
19     ms.x = ms.y;
20     ms.y = temp;
21 }
22
23 int main()
24 {
25     myStructure ms = { 50, 60 };
26
27     cout<<"\n\nBefore swap, inside main():\n";
28     cout<<"x = "<<ms.x<<"\ty = "<<ms.y;
29
30     swap(ms);
31
32     cout<<"\n\nAfter swap, inside main():\n";
33     cout<<"x = "<<ms.x<<"\ty = "<<ms.y;
34     cout<<endl;
35     return 0;
36 }
37

```

Before swap, inside main():
x = 50 y = 60

After swap, inside main():
x = 60 y = 50

```
#include <iostream.h>
void get_value( int &j );
int main()
{
    int i = 0 ;
    cout << "before call i = " << i << endl ;
    get_value( i ) ;
    cout << "after call i = " << i << endl ;
    return 0;
}
```

```
void get_value( int &j )
{
    cout << "value passed to func = " << j << endl ;
    cin >> j ;
    cout << "value at end func = " << j << endl ;
}
```

```
before call i = 0
value passed to func = 0
13
value at end func = 13
after call i = 13
```

```
#include <iostream.h>
void j2Mul ( int &j );
int main()
{
    int i = 1 ;
    cout << "before call i = " << i << endl ;
    j2Mul( i ) ;
    cout << "after call i = " << i << endl ;
    j2Mul( i ) ;
    cout << "after call i = " << i << endl ;
    return 0;
}
```

What the value of i , j
after main function call
j2Mul function are ?

```
void j2Mul ( int &j )
{
    cout << "value at being func = " << j << endl ;
    j = j*20;
    cout << "value at end func = " << j << endl ;
}
```



Array & Function in C/C++

□ “pass by reference”

Pass array to function (pass by reference)

main.cpp

```
2 using namespace std;
3 void updateArray( int data[5] )
4 {
5     for(int i = 0 ; i < 5 ; i++) {
6         data[i] = data[i]+1000;
7     }
8 }
9 void showarray( int data[5] )
10 {
11     for(int i = 0 ; i < 5 ; i++) {
12         cout << data[i] << ",";
13     }
14 }
15 int main() {
16
17     int Roll_Number[5] = {100,101,102,103,104};
18
19     cout << "\nbefore update \n";
20     showarray(Roll_Number);
21     updateArray(Roll_Number);
22     cout << "\nafter update \n";
23     showarray(Roll_Number);
24 }
25
```

```
before update
100,101,102,103,104,
after update
1100,1101,1102,103,104,
```

Pass array to function (pass by reference)

```
1  #include <iostream>
2  using namespace std;
3  void updateArray( int data[ ],int n )
4  {
5      for(int i = 0 ; i < n ; i++) {
6          data[i] = data[i]+1000;
7      }
8  }
9  void showarray( int data[ ],int n )
10 {
11     for(int i = 0 ; i < n ; i++) {
12         cout << data[i] << ",";
13     }
14 }
15 int main() {
16
17     int Roll_Number1[5] = {100,101,102,103,104};
18     int Roll_Number2[5] = {1000,1010,1020};
19
20     cout << "\nbefore update Roll_Number1 \n";
21     showarray(Roll_Number1,5);
22     updateArray(Roll_Number1,3);
23     cout << "\nafter update \n";
24     showarray(Roll_Number1,5);
25
26     cout << "\nshow Roll_Number2\n";
27     showarray(Roll_Number2,3);
28 }
29
30
```

Not allow to return Array

main.cpp

```
1  #include <iostream>
2  using namespace std;
3  //return pointer data type
4  int * returnArray( )
5  {
6      int myarray[100];
7      return myarray;
8  }
9  //return pointer data type
10 int *returnPassArray( int passData[ ] )
11 {
12     return passData;
13 }
14 int main() {
15
16     int myData1[100];
17     myData1 = returnArray();
18
19     int myData2[100];
20     myData2 = returnPassArray( myData1);
21
22 }
23
```

Pass array of characters or string



```
main.cpp
1  //C++
2  //include <iostream>
3  //using namespace std;
4
5  //TurboC
6  #include <iostream.h>
7
8  void showarray( char *data )
9  {
10     cout << "data -> " << data;
11     cout << "\n";
12 }
13 void updateArray( char data[ ] )
14 {
15     showarray(data);
16     for(int i = 0 ; data[i] != '\0' ; i++) {
17         data[i] = data[i]+1;
18     }
19     showarray(data);
20 }
21
22 int main() {
23     //array of charector or string
24     char mystring1[] = "012345678";
25     updateArray(mystring1);
26
27     //string constant
28     char *mystring2 = "abcdef";
29     updateArray(mystring2);
30     cout << "end ";
31 }
32
33
```



Overload function

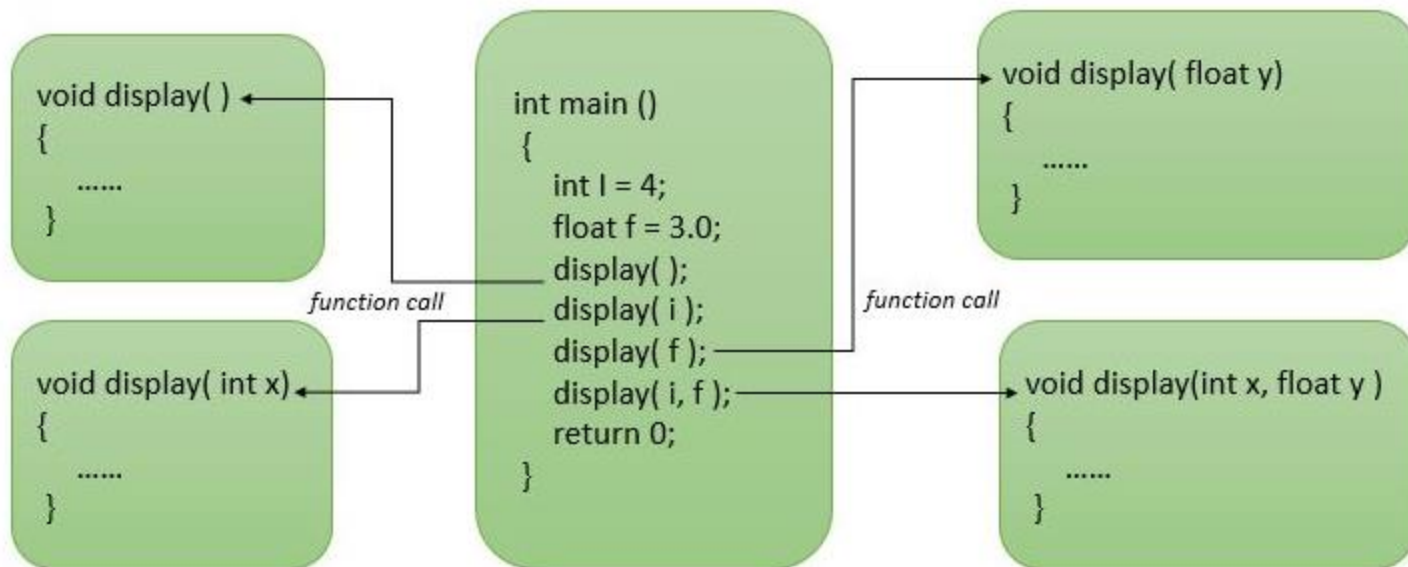
improve readability

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  void print_int(int i) {
5      cout << " Here is int " << i << endl;
6  }
7  void print_double(double f) {
8      cout << " Here is float " << f << endl;
9  }
10 void print_array(char *c) {
11     cout << " Here is char* " << c << endl;
12 }
13
14 int main() {
15     print_int(10);
16     print_double(10.10);
17     print_array("ten");
18     return 0;
19 }
20
21
```

Overload function

Illustration of C++ function overloading



main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  void print(int i) {
5      cout << " Here is int " << i << endl;
6  }
7  void print(double f) {
8      cout << " Here is float " << f << endl;
9  }
10 void print(char *c) {
11     cout << " Here is char* " << c << endl;
12 }
13
14 int main() {
15     print(10);
16     print(10.10);
17     print("ten");
18     return 0;
19 }
20
21
```



More about Function Prototypes

- A function prototypes consists of
 - The function's return type.
 - Function Name
 - argument list
- However, variable's name of argument list may ignore.

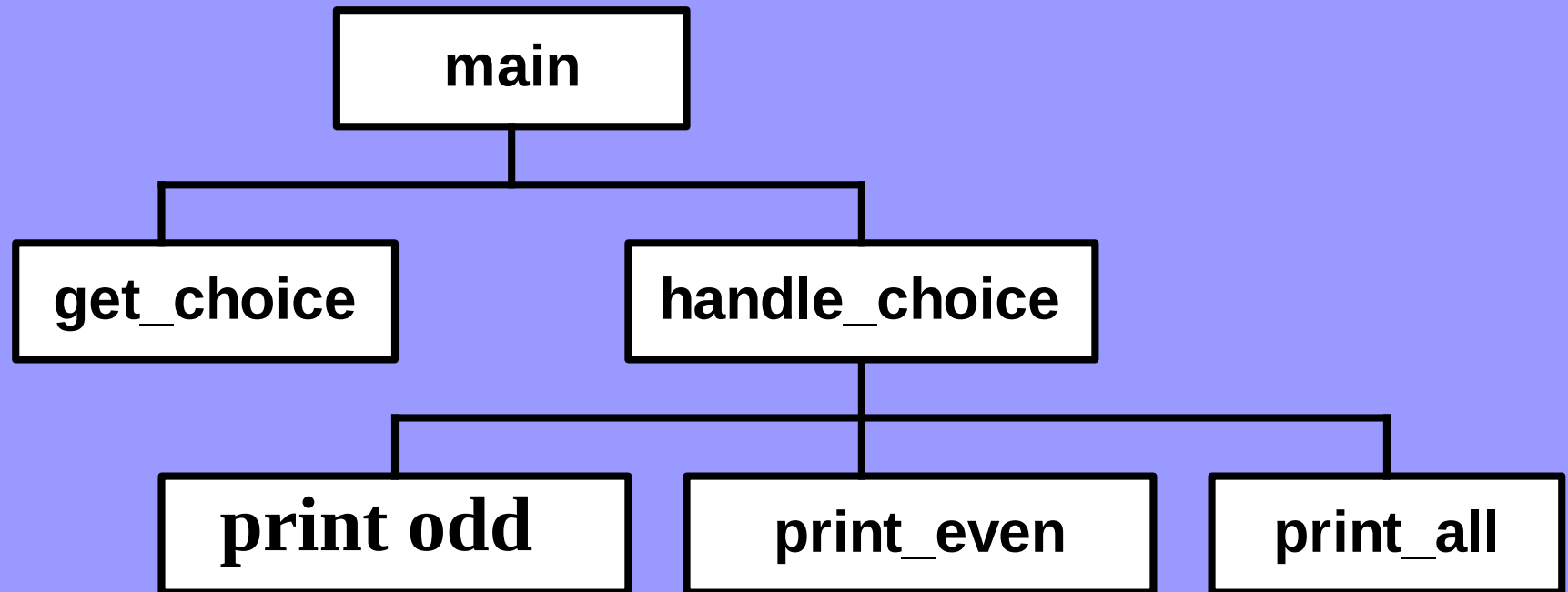
NOTE : *More about Function Prototypes in C/C++*
variable's name of argument list may ignore.

```
//double celsius_to_fahrenheit( double celsius );  
double celsius_to_fahrenheit( double );
```

```
double celsius_to_fahrenheit( double celsius )  
{  
    double fahr ;  
    fahr = celsius * ( 9.0/5.0) + 32.0 ;  
    return fahr ;  
}
```

```
//void Swap ( int &x , int &y ) ;  
void Swap ( int & , int & ) ;  
void Swap ( int &x ,int &y )  
{  
    int temp ;  
    temp = x ;  
    x = y ;  
    y = temp ;  
}
```

Rewrite function using pass by reference



```
▲ /tmp/MmMbJeNdMr.o
Which series do you wish to display?
1 - Odd numbers from 1 to 30
2 - Even numbers from 1 to 30
3 - All numbers from 1 to 30
|
```

Previous version

```
1 #include <iostream>
2 using namespace std;
3
4 int getchoice() {
5     int choice; // variable for user input
6     do // loop until a valid choice is entered
7     {
8         cout << "Which series do you wish to display?\n";
9         cout << "1 - Odd numbers from 1 to 30\n";
10        cout << "2 - Even numbers from 1 to 30\n";
11        cout << "3 - All numbers from 1 to 30\n";
12        cin >> choice; // get choice from user
13        if ((choice < 1) || (choice > 3))
14        { // if invalid entry, give message
15            cout << "Choice must be 1, 2, or 3\n";
16        }
17    } while ((choice < 1) || (choice > 3));
18    return choice;
19 }
20
21 void printOdd() {
22     for (int i = 1; i <= 30; i = i + 2)
23         cout << i << ' ';
24     cout << endl;
25 }
26
27 void printEvent() {
28     for (int i = 2; i <= 30; i = i + 2)
29         cout << i << ' ';
30     cout << endl;
31 }
32
33 void printAll() {
34     for (int i = 2; i <= 30; i = i + 2)
35         cout << i << ' ';
36     cout << endl;
37 }
38
39 int main()
40 {
41
42     int choice = getchoice();
43     switch (choice)
44     {
45         case 1:
46             printOdd();
47             break;
48         case 2:
49             printEvent() ;
50             break;
51         case 3:
52             printAll( );
53             break;
54     }
55     return 0;
56 }
57
```

Rewrite program with procedural concept

แบบฝึกหัด 2

จงเขียนโปรแกรมต้องการนำทะเบียนลูกค้า(customer)
ของสหกรณ์ออมทรัพย์รามคำแหง ที่รองรับการจัดเก็บข้อมูล
ไม่เกิน 500 คน ทะเบียนลูกค้าประกอบด้วย รหัสบัญชี(id)
ชื่อบัญชี(name) ที่อยู่(Address) และเงินฝาก
(Deposit)

```
Interest = deposit*10%;  
Deposist = Deposit+interst
```

Menu

1. Open a new Account
2. Deposit
3. Withdraw
4. Compound interest
5. Quit

Please select 1 / 2 / 3 / 4 / 5 =?



main

Opennewacc

widthdraw

report

menu

deposit

Compond
interest

```

1469 int main()
1470 {
1471     ....struct person customer[500];
1472     ....int n=0;
1473     ....int choice;
1474     ....char id[5];
1475     ....float _deposit, _widthdraw;
1476     ....char name[100];
1477     ....char addr[100];
1478     ....int res;
1479     ....choice = displaymenu();
1480     ....while(choice!=6){
1481         ....switch(choice){
1482             ....case 1:
1483                 ..../*
1484                 —> —> —> int opennewacc(char id[5],
1485                 —> —> —> ....float deposit, char name[], char addr[]
1486                 —> —> —> ....int &n, struct person p[]) .....
1487                 ....*/
1488                 ....cout << "id: ";
1489                 ....cin >> id;
1490                 ....cout << "name: ";
1491                 ....cin >> name;
1492                 ....cout << "addr: ";
1493                 ....cin >> addr;
1494                 ....cout << "deposit: ";
1495                 ....cin >> _deposit;
1496                 ....res = opennewacc(id,
1497                 ....    _deposit,
1498                 ....    name,
1499                 ....    addr,
1500                 ....    n,
1501                 ....    customer);
1502                 ....if(res==0){
1503                 ....    cout << "accout duplicate error" << endl;
1504                 ....}
1505                 ....break;

```

```

1506 .....case 2:
1507 ...../*
1508 .....int deposit(char id[5],float m,int n,struct person p[])
1509 .....*/
1510 .....cout<<"id: ";
1511 .....cin>>id;
1512 .....cout<<"deposit: ";
1513 .....cin>>_deposit;
1514 .....res=deposit(id,_deposit,n,customer);
1515 .....
1516 .....break;
1517 .....case 3:
1518 ...../*
1519 .....int widthdraw(char id[5],float m,int n,struct person p[])
1520 .....*/
1521 .....cout<<"id: ";
1522 .....cin>>id;
1523 .....cout<<"widthdraw: ";
1524 .....cin>>_widthdraw;
1525 .....res=widthdraw(id,_widthdraw,n,customer);
1526 .....break;
1527 .....case 4:
1528 .....break;
1529 .....
1530 .....case 5:
1531 .....report(n,customer);
1532 .....break;
1533 .....}
1534 .....choice=displaymenu();
1535 .....}
1536 .....
1537 .....return 0;
1538 }
1539

```

```

1366 #include <iostream>
1367 #include <string.h>
1368 #include <iomanip>
1369
1370 using namespace std;
1371
1372 struct person {
1373     char id[5];
1374     char name[30];
1375     char addrs[50];
1376     float deposit;
1377 };
1378 int displaymenu()
1379 {
1380     int choice;
1381     do {
1382         cout << "1.open acc" << endl;
1383         cout << "2.deposit acc" << endl;
1384         cout << "3.withdraw acc" << endl;
1385         cout << "4.compond interest acc" << endl;
1386         cout << "5.reports " << endl;
1387         cout << "6.quit" << endl;
1388         cin >> choice;
1389     } while ( (choice >= 1 && choice <= 5) == false );
1390     return choice;
1391 }

```

```

1393 int checkaccExist(char id[5], int n, struct person p[])
1394 { // 0..n-1 found, -1=not found
1395     for(int i=0; i<n; i++) {
1396         if(strcmp(id, p[i].id)==0) {
1397             return i;
1398         }
1399     }
1400     return -1;
1401 }
1402 int opennewacc(char id[5],
1403     float deposit, char name[], char addr[],
1404     int &n, struct person p[])
1405 {
1406     int res = checkaccExist(id, n, p); // 0..n-1 found, -1=not found
1407     if(res >= 0) {
1408         return 0;
1409     }
1410     /*
1411     struct person {
1412         char id[5];
1413         char name[30];
1414         char addr[50];
1415         float deposit;
1416     } */
1417     /*
1418     strcpy(p[n].id, id);
1419     strcpy(p[n].name, name);
1420     strcpy(p[n].addr, addr);
1421     p[n].deposit = deposit;
1422     n++;
1423     return 1;
1424     // 1 success
1425     // 0 not success
1426     */
1427 int deposit(char id[5], float m, int n, struct person p[]) {
1428     // 1 success
1429     // 0 not success
1430     int idex = checkaccExist(id, n, p); // 0..n-1 found, -1=not found
1431     if(idex < 0) {
1432         return 0;
1433     }
1434     p[idex].deposit += m;
1435     return 0;
1436 }

```

```

1438 int widthdraw(char id[5],float m,int n,struct person p[]) {
1439 //1.success
1440 //0.not success
1441 ....int idex.=.checkaccExist(id,n,p);. //0..n-1.found,-1=not.found
1442 ....if(.idex.<.0) {
1443 .....return 0;
1444 ....}
1445 ....if(p[idex].deposit.-m.<.100.) {
1446 .....return 0;
1447 ....}
1448 ....p[idex].deposit.-=m;
1449 ....return 1;
1450 }
1451 void compondinterest(.float rate,int n,struct person p[]) {
1452 }
1453
1454 void report(.int n,struct person p[]) {
1455 ....for(int i.=.0; i.<.n; i++) {
1456 .....cout.<< "id : " << setw(10) ;.
1457 .....cout.<< p[i].id.<< setw(10) ;.
1458 .....cout.<< "name : " << setw(10) ;.
1459 .....cout.<< p[i].name.<< setw(10) ;.
1460 .....cout.<< "addrs : " << setw(10) ;.
1461 .....cout.<< p[i].addrs.<< setw(10) ;.
1462 .....cout.<< "deposit : " << setw(10) ;.
1463 .....cout.<< p[i].deposit<< setw(10) ;.
1464 .....cout.<< endl;
1465 ....}
1466 }

```

ตัวอย่าง

ให้นักศึกษาเขียนโปรแกรมขายสินค้าและพิมพ์ใบเสร็จรับเงิน
โดยข้อมูลในระบบมีไม่เกิน 100 รายการ
ข้อมูลขายสินค้าด้วย

- รหัสสินค้า(id) เช่น “AB001”
- ชื่อสินค้า(name) เช่น “CAFÉ”
- ราคาขาย(cost) เช่น 30.6
- จำนวนสินค้าในสต็อก (instock) เช่น 5

Menu

1. Add item to Cart
2. Sell Product
3. Check Stock
4. End Program

Please select 1 / 2 / 3/ 4 =?

Rewrite your code with
Function approach

BillStructure.cpp

Apirak :



Rewrite the code using function approach



ตัวอย่าง

ให้นักศึกษาเขียนโปรแกรมขายสินค้าและพิมพ์ใบเสร็จรับเงิน
โดยข้อมูลในระบบมีไม่เกิน 100 รายการ
ข้อมูลขายสินค้าด้วย

- รหัสสินค้า(id) เช่น “AB001”
- ชื่อสินค้า(name) เช่น “CAFÉ”
- ราคาขาย(cost) เช่น 30.6
- จำนวนสินค้าในสต็อก (instock) เช่น 5

Menu

1. Add item to Cart
2. Sell Product
3. Check Stock
4. End Program

Please select 1 / 2 / 3/ 4 =?

กด 1

Input your product data:

id : **AB001**

name : **CAFE**

Cost : **30.6**

Stock : **5**

กด 2

Input your product id: **AB001**

Input your sell volume : **2**

name : **CAFE**

Cost : **30.6**

stock : **3**

Pay : 61.2 bath

Apirak :

Menu

1. Add item to Cart
2. Sell Product
3. Check Stock
4. End Program

Please select 1 / 2 / 3/ 4 =?

กด 3

product List:

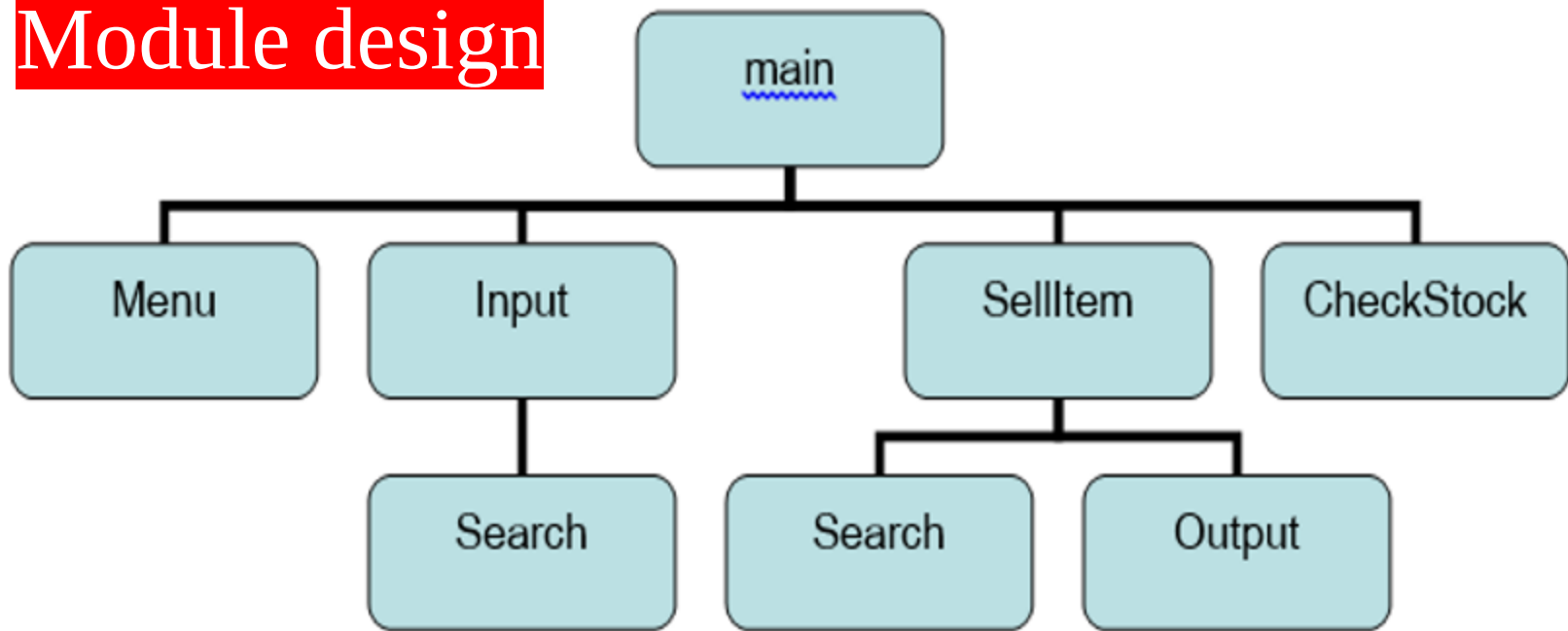
Id	name	cost	stock
----	------	------	-------

AB001	CAFÉ	30.6	3
-------	------	------	---

กด 4

exit from program

Module design



- EX1** จากตัวอย่างข้างต้นให้ นักศึกษาปรับโปรแกรม โดยให้มีการตรวจดังนี้
- กรณี กค 1 : ให้ตรวจสอบว่ารหัสที่ป้อน มีซ้ำหรือไม่ ถ้าไม่มีจึงจะอนุญาตให้ผู้ใช้ป้อนข้อมูล
- กรณี กค 2: ให้ตรวจสอบว่ารหัสสินค้าที่ผู้ใช้ป้อนมีอยู่ในระบบหรือไม่ กรณีไม่พบให้แสดงข้อความ “product not exist”
- กรณี กค 3 ให้แสดงรายการทั้งหมด และแสดงยอดรวมของ cost



7.3 Using Library Functions

- C++ provide a lot of standard functions, called *library functions for programmer*.
- The prototypes of library functions are defined to your program using `#include` compiler directive.



```
#include <iostream.h>
```

```
#include <math.h>
```

```
int main(void)
```

```
{
```

```
    double x,y ,z ;
```

```
    cout << "get x= "; cin >> x ;
```

```
    cout << "get y= "; cin >> y ;
```

```
    z = pow( x, y );
```

```
    cout << "get x**y = " << z ;
```

```
    return 0;
```

```
}
```



Popular Math Function

Function	Prototype	Description
abs	int abs(int x)	return absolute value of an int
labs	long abs(long int x)	return absolute value of long int
fabs	double abs(double x)	return absolute value of double
ceil	double ceil(double x)	rounds up to a whole number
floor	double floor(double x)	rounds down to a whole number
pow	double pow(double x,double y)	calculate x to the power of y
pow10	double pow10(int x)	calculate 10 to the power of y
sqrt	double sqrt(double x)	calculate the positive square root of x



Functions for Working with character

- C++ also provide many functions for analyzing and changing characters.

Function	Prototype	Description
isupper	int isupper(char x)	Determines if a character is upper
islower	int islower(char x)	Determines if a character is lower
isalpha	int isalpha(char x)	Determines if a character is 'a'-'z'
isdigit	int isdigit(char x)	Determines if a character is '0'-'9'
toupper	int toupper(char x)	Converts a character to upper
tolower	int tolower(char x)	Converts a character to lower

Run Debug Stop Share

main.cpp

```
1 #include <iostream>
2 using namespace std;
3 #include <ctype.h>
4 int main()
5 {
6     char c;
7     cout << "enter a characer :";
8     cin >> c;
9     if( isalpha(c) )
10     {
11         if( isupper(c) )
12             { cout << c << " is an upper leter\n"; }
13         else
14             { cout << c << " is an lower leter\n"; }
15     }
16     if( isdigit(c) )
17         { cout << c << " is a number\n"; }
18     if( !( isdigit(c) || isalpha(c) ) )
19         { cout << c << " is a number\n"; }
20 }
21
```

```
C:\My Documents\sc107>charfun
enter a characer :a
a is an lower leter
```

```
C:\My Documents\sc107>charfun
enter a characer :1
1 is a number
```



Design methodology

- ***Top-down design:** begins with the function at top of diagram and works toward the function at the bottom of the diagram.*
- ***Bottom-up design:** involves begins with the bottom of diagram and working your way up.*

ตัวอย่าง (3)

จงเขียนโปรแกรมอ่านข้อมูล 2 ชนิด คือ **A** และ **B** ซึ่งชนิดละ 20 จำนวน จากนั้นให้คำนวณค่า

$$A_bar = \text{Sum } i = 1 .. 20 \text{ ของ } A$$

$$B_bar = \text{Sum } i = 1 .. 20 \text{ ของ } B$$

Corr(A , B) =

$$\frac{n * \text{Sum}(\underline{AiBi}) - \text{Sum}(Ai) * \text{Sum}(Bi)}{$$

$$\text{sqrt} (n * (\text{sum}(Ai * Ai) - \text{Sum}(Ai) * \text{Sum}(Ai)) *$$

$$n * (\text{sum}(Bi * Bi) - \text{Sum}(Bi) * \text{Sum}(Bi)))$$

กำหนดให้ $z = \text{sqrt}(x)$ เป็น build in function ของการหาค่า root x

```

4
5 void GetData ( float Z[ ] ,int n );
6 float Sum ( float Z[ ] , int n ) ;
7 void MultAry ( float W[ ] , float X[ ] , float Z[ ] , int n ) ;
8
9
10 int main( )
11 {
12     float A [ 20 ] , B[ 20 ] ,T[20] ;
13     float A_bar , B_bar ,corrAB , X , Y;
14     float AB_bar , AA_bar , BB_bar ;
15     cout << " input DATA of Matrix A \n" ;
16     GetData ( A ,20 ) ;
17     cout << " input DATA of Matrix B \n" ;
18     GetData ( B , 20 ) ;
19     A_bar = Sum ( A , 20 ) ;
20     B_bar = Sum ( B , 20 ) ;
21     MultAry( A , A , T , 20 ) ; AA_bar = Sum ( T, 20 ) ;
22     MultAry( B , B , T , 20 ) ; BB_bar = Sum ( T, 20 ) ;
23     MultAry( A , B , T , 20 ) ; AB_bar = Sum ( T, 20 ) ;
24
25     X = (20*AB_bar - A_bar*B_bar) ;
26     Y = ( ( 20* (AA_bar - A_bar* A_bar) ) *
27           ( 20* (BB_bar - B_bar* B_bar) ) );
28     corrAB = X / sqrt ( Y ) ;
29
30     cout << "\ncorr(A,B)" << corrAB ;
31
32     return 0;
33 }
34

```

สมมติว่า Apartment แห่งหนึ่ง มีจำนวนห้องพัก 100 ห้อง จงเขียนโปรแกรม
คำนวณค่าปริมาณการใช้น้ำดังนี้

1. โปรแกรมแสดงเมนู

1. add meter

2. Caculate meter

3. Diplay meter

2. ข้อมูลที่รับเข้าประกอบด้วย

1. เลขประจำมิเตอร์ (no. of meter) จำนวน 4 ตัวอักษร เช่น A001,A002

2. ค่าม (start meter value)

3.ค่ามิเตอร์



Best practice : Guide for coding subroutine

- ❑ Get single value
- ❑ Pass value and get value
- ❑ Return multiple value using pass by reference
- ❑ return local array

Return function vs pass by ref

```
2  #include <iostream>
3
4  using namespace std;
5
6  int getonedata() {
7      int i;
8      cout << " enter data : " ;
9      cin>> i;
10     return i;
11 }
12 int main()
13 {
14     int x = getonedata();
15     return 0;
16 }
17
```

```
1
2  #include <iostream>
3
4  using namespace std;
5
6  void getonedata(int &i) {
7      cout << " enter data : " ;
8      cin>> i;
9  }
10 int main()
11 {
12     int x;
13     getonedata(x);
14     return 0;
15 }
16
```

Return function vs pass by ref

```
2 #include <iostream>
3 using namespace std;
4
5 int sumx() {
6     return 10;
7 }
8
9 int main()
10 {
11     int x1,x1,s1;
12     x1 = sumx();
13     x2 = sumx();
14     s1 = x1 + x2;
15     cout << "s1 = " << s1;
16
17     int s2 = sumx() + sumx();
18     cout << "s2 = " << sum;
19
20     return 0;
21 }
```

```
1
2 #include <iostream>
3 using namespace std;
4
5 void sumx(int &x) {
6     x = 10;
7 }
8
9 int main()
10 {
11     int x1,x1,s1;
12     sumx(x1);
13     sumx(x2);
14     s1 = x1 + x2;
15     cout << "s1 = " << s1;
16
17     //int s2 = sumx(x1) + sumx();
18     //cout << "s2 = " << sum;
19
20     return 0;
21 }
```



main.cpp

```
1  #include <iostream>
2  using namespace std;
3  void f1(int &x) {
4      x = 10;
5  }
6  void f2(int &x )
7  {
8      x = x+10;
9  }
10 int main()
11 {
12     int x1,x2,x3,x4;
13
14     f1(x1);
15     x2 = 0;
16     f1(x2);
17
18     f2(x3);
19     x4 = 20;
20     f2(x4);
21     return 0;
22 }
23
```

Problem?

Get single value

```
2 #include <iostream>
3
4 using namespace std;
5
6 int getonedata() {
7     int i;
8     cout << " enter data : " ;
9     cin>> i;
10    return i;
11 }
12 int main()
13 {
14     int x = getonedata();
15     return 0;
16 }
17
```

```
1
2 #include <iostream>
3
4 using namespace std;
5
6 void getonedata(int &i) {
7     cout << " enter data : " ;
8     cin>> i;
9 }
10 int main()
11 {
12     int x;
13     getonedata(x);
14     return 0;
15 }
16
2 #include <iostream>
3
4 using namespace std;
5
6 void getonedata(int &i) {
7     cout << " enter data : " ;
8     cin>> i;
9 }
10 int main()
11 {
12     int x=30;
13     getonedata(x);
14     return 0;
15 }
16
```

Pass value and get value

```
1
2 #include <iostream>
3
4 using namespace std;
5
6 void dosomething2(int &i) {
7     i = i*3;
8 }
9 int main()
10 {
11     int x=30;
12     dosomething2(x);
13     cout << x;
14     return 0;
15 }
```

```
1 #include <iostream>
2
3 using namespace std;
4
5 void dosomething1(int x,int &i) {
6     i = x*3;
7
8 }
9 int main()
10 {
11     int x=30;
12     dosomething1(x, x);
13     cout << x;
14     return 0;
15 }
```

Return multiple value using pass by reference (1)

```
main.cpp
1  #include <iostream>
2
3  using namespace std;
4
5  void dosomething3(int &i,int &j ,int &k) {
6      cin >> i;
7      cin >> j;
8      cin >> k;
9  }
10 int main()
11 {
12     int a,b,c;
13     dosomething3(a,b,c);
14     return 0;
15 }
16
```

```
main.cpp
1  #include <iostream>
2
3  using namespace std;
4
5  int dosomething4(int &i,int &j) {
6      cin >> i;
7      cin >> j;
8      cin >> k;
9      return k;
10 }
11 int main()
12 {
13     int a,b,c;
14     c = dosomething4(a,b);
15     return 0;
16 }
17
18
```

Return multiple value

using pass by reference (2)

```
1 #include <iostream>
2
3 using namespace std;
4
5 void dosomething3(int &i,int &j ,int &k) {
6     cin >> i;
7     cin >> j;
8     cin >> k;
9 }
10 int main()
11 {
12     int a,b,c;
13     dosomething3(a,b,c);
14     return 0;
15 }
16
```

```
1 #include <iostream>
2
3 using namespace std;
4
5 int dosomething5(int &i,int &j,int &k) {
6     cin >> i;
7     cin >> j;
8     cin >> k;
9     return 0;
10 }
11 int main()
12 {
13     int a,b,c;
14     if(dosomething5(a,b,c)==0)
15         cout << "sucess";
16     else
17         cout << "fail";
18     return 0;
19 }
20
```

Return status of function

Passing Array 2 Dimension in C/C++

main.cpp

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  void GetData1 ( int Z [ 3 ] [ 3 ] )
6  {
7      int i , j ;
8      cout << "GetData1 ( int Z [ 3 ] [ 3 ] ) \n";
9      for ( i = 0 ; i < 3 ; i++ )
10     {
11         for ( j = 0 ; j < 3 ; j++ )
12         {
13             Z[ i ] [ j ] = rand()%10;
14             cout << setw(5) << Z[ i ] [ j ] << " ";
15         }
16         cout << endl;
17     }
18 }
19
20 void GetData2 ( int Z [ ] [ 3 ] ,int numrow)
21 {
22     int i , j ;
23
24     cout << " ( int Z [ ] [ 3 ] ,int numrow)" << numrow << "\n";
25     for ( i = 0 ; i < numrow ; i++ )
26     {
27         for ( j = 0 ; j < 3 ; j++ )
28         {
29             Z[ i ] [ j ] = rand()%10;
30             cout << setw(5) << Z[ i ] [ j ] << " ";
31         }
32         cout << endl;
33     }
34 }
```

```
35  /*
36  void GetData3 ( int Z [ 3 ] [ ] )
37  {
38      int i , j ;
39
40      for ( i = 0 ; i < 3 ; i++ )
41      {
42          for ( j = 0 ; j < 3 ; j++ )
43          {
44              Z[ i ] [ j ] = rand()%10;
45              cout << setw(5) << Z[ i ] [ j ] << " ";
46          }
47          cout << endl;
48      }
49  }
50  */
51
52 int main()
53 {
54     int A[3][3];
55     int B[5][3];
56     GetData1(A);
57     GetData2(B,5);
58     return 0;
59 }
```



ตัวอย่าง แบบ top down

จงเขียนโปรแกรม**รับข้อมูล**ลงอาร์เรย์ A , B ขนาด 20x20
และ**หาผลคูณ**ของอาร์เรย์ C โดย

$$C = A \times B$$

สูตรการคูณ คือ $C_{ij} = \sum a_{ik} b_{kj}$

เมื่อ $1 \leq i \leq 20$ และ $1 \leq j \leq 20$



Break down the problem

1. Get data both A and B
2. Calculate $C = A * B$;
3. Show C

```

1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  void GetData ( int Z [3 ] [3] )
6  {
7      int i , j ;
8
9      for ( i = 0 ; i < 3 ; i++ )
10     {
11         for ( j = 0 ; j < 3 ; j++ )
12         {
13             //cout << " row = " << i << "column " << j ;
14             //cin >> Z[ i ] [ j ] ;
15             // Get a random number
16             Z[ i ] [ j ] = rand()%10;
17             cout << setw(5) << Z[ i ] [ j ] << " ";
18         }
19         cout << endl;
20     }
21 }
22
23 void MultiplyMatrix ( int W[ 3 ] [3 ],
24                      int X[3][3],int Z[3][3 ] )
25 {
26     int i , j , k ,sum ;
27     for ( i = 0 ; i < 3 ; i++ )
28     {
29         for ( j = 0 ; j < 3 ; j++ )
30         {
31             sum = 0 ;
32             for ( k = 0 ; k < 3 ; k++ )
33             {
34                 sum =sum + W[ i ] [ k ] * X [ k ] [ j ] ;
35             }
36             Z[ i ] [ j ] = sum ;
37         }
38     }
39 }
40

```

```

40
41 void PrintMatrix( int Z [3 ] [3] )
42 {
43     int i , j ;
44     cout << "value of Matrix C \n" ;
45     for ( i = 0 ; i < 3 ; i++ )
46     {
47         for ( j = 0 ; j < 3 ; j++ )
48         {
49             cout << setw(5) << Z[ i ] [ j ] << " ";
50         }
51         cout << endl;
52     }
53 }
54 int main( )
55 {
56     int A[ 3 ] [ 3 ] ;
57     int B[ 3 ] [3] ;
58     int C[ 3 ] [ 3 ] ;
59     int i , j ;
60
61     cout << "input DATA of Matrix A \n" ;
62     GetData ( A ) ;
63     cout << "input DATA of Matrix B \n" ;
64     GetData ( B ) ;
65     MultiplyMatrix ( A, B , C ) ;
66     PrintMatrix(C);
67 }
68

```

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  void GetData ( int Z [ ] [3] )
6  {
7      int i , j ;
8
9      for ( i = 0 ; i < 3 ; i++ )
10     {
11         for ( j = 0 ; j < 3 ; j++ )
12         {
13             //cout << " row = " << i << "column " << j ;
14             //cin >> Z[ i ] [ j ] ;
15             // Get a random number
16             Z[ i ] [ j ] = rand()%10;
17             cout << setw(5) << Z[ i ] [ j ] << " ";
18         }
19         cout << endl;
20     }
21 }
22
```

```
23 void MultiplyMatrix ( int W[ ] [3],
24                       int X[ ][3],int Z[][3] )
25 {
26     int i , j , k ,sum ;
27     for ( i = 0 ; i < 3 ; i++ )
28     {
29         for ( j = 0 ; j < 3 ; j++ )
30         {
31             sum = 0 ;
32             for ( k = 0 ; k < 3 ; k++ )
33             {
34                 sum =sum + W[ i ] [ k ] * X [ k ] [ j ] ;
35             }
36             Z[ i ] [ j ] = sum ;
37         }
38     }
39 }
40
```



```
40
41 void PrintMatrix( int Z [ ] [3] )
42 {
43     int i , j ;
44     cout << "value of Matrix C \n" ;
45     for ( i = 0 ; i < 3 ; i++ )
46     {
47         for ( j = 0 ; j < 3 ; j++ )
48         {
49             cout << setw(5) << Z[ i ] [ j ] << " ";
50         }
51         cout << endl;
52     }
53 }
54 int main( )
55 {
56     int A[ 3 ] [ 3 ] ;
57     int B[ 3 ] [3] ;
58     int C[ 3 ] [ 3 ] ;
59     int i , j ;
60
61     cout << "input DATA of Matrix A \n" ;
62     GetData ( A ) ;
63     cout << "input DATA of Matrix B \n" ;
64     GetData ( B ) ;
65     MultiplyMatrix ( A, B , C ) ;
66     PrintMatrix(C);
67 }
68
69
```

ตัวอย่าง (3)

จงเขียนโปรแกรมอ่านข้อมูล 2 ชนิด คือ **A** และ **B** ซึ่งชนิดละ 20 จำนวน จากนั้นให้คำนวณค่า

$$A_bar = \text{Sum } i = 1 .. 20 \text{ ของ } A$$

$$B_bar = \text{Sum } i = 1 .. 20 \text{ ของ } B$$

Corr(A , B) =

$$\frac{n * \text{Sum}(\underline{AiBi}) - \text{Sum}(Ai) * \text{Sum}(Bi)}{\text{sqrt} (n * (\text{sum}(Ai * Ai) - \text{Sum}(Ai) * \text{Sum}(Ai)) * }$$

$$n * (\text{sum}(Bi * Bi) - \text{Sum}(Bi) * \text{Sum}(Bi)))$$

กำหนดให้ $z = \text{sqrt}(x)$ เป็น build in function ของการหาค่า root x

```

4
5 void GetData ( float Z[ ] ,int n );
6 float Sum ( float Z[ ] , int n ) ;
7 void MultAry ( float W[ ] , float X[ ] , float Z[ ] , int n ) ;
8
9
10 int main( )
11 {
12     float A [ 20 ] , B[ 20 ] ,T[20] ;
13     float A_bar , B_bar ,corrAB , X , Y;
14     float AB_bar , AA_bar , BB_bar ;
15     cout << " input DATA of Matrix A \n" ;
16     GetData ( A ,20 ) ;
17     cout << " input DATA of Matrix B \n" ;
18     GetData ( B , 20 ) ;
19     A_bar = Sum ( A , 20 ) ;
20     B_bar = Sum ( B , 20 ) ;
21     MultAry( A , A , T , 20 ) ; AA_bar = Sum ( T, 20 ) ;
22     MultAry( B , B , T , 20 ) ; BB_bar = Sum ( T, 20 ) ;
23     MultAry( A , B , T , 20 ) ; AB_bar = Sum ( T, 20 ) ;
24
25     X = (20*AB_bar - A_bar*B_bar) ;
26     Y = ( ( 20* (AA_bar - A_bar* A_bar) ) *
27         ( 20* (BB_bar - B_bar* B_bar) ) );
28     corrAB = X / sqrt ( Y ) ;
29
30     cout << "\ncorr(A,B)" << corrAB ;
31
32     return 0;
33 }
34

```

<https://www.w3resource.com/cpp-exercises>

END



One dimensional array

ตัวอย่าง 3

```
//C++ method of passing Array to function is
//passing by reference
void GetData( float A [ ] , int n ) ;
{
    int i ;
    for ( i = 0 ; i < n ; i++)
    {
        cin >> A[ i ] ;
    }
}
```

One dimensional array

ตัวอย่าง 3

```
//C++ method of passing Array to function is  
//passing by reference  
float Sum( float X [ ] , int n ) ;  
{  
    int i ;  
    float s = 0 ;  
    for ( i = 0 ; i < n ; i++)  
    {  
        s = s + X [ i ] ;  
    }  
    return s ;  
}
```

```
void MultAry ( float W[ ], float X[ ], float Z[ ], int n ) ;  
{  
    int i ;  
    for ( i = 0 ; i < n ; i++)  
    {  
        Z[ i ] = W [ i ] * X [ i ] ;  
    }  
    return ;  
}
```



Passing One dimensional Array to function

One dimensional array

ตัวอย่าง 3

จงเขียนโปรแกรมหาค่าสูงสุดและต่ำสุด และตำแหน่ง
ของค่าสูงและต่ำสุดของอาร์เรย์ Z ขนาด 120 เข้าสู่
อาร์เรย์ Z

One dimensional array

ตัวอย่าง 3

```
void GetData( int A [120] ) ;  
void main( )  
{  
    int z[120] , i ;  
    int max , min ;  
    int imax , imin ;  
  
    GetData ( Z ) ;  
  
    max = z[0] ; min = z[0];  
    imax = 0;    imin = 0;
```

```
    for ( i = 1 ; i < 120 ; i++)  
    {  
        if (z[ i ] > max ) {  
            max = z[ i ] ;  
            imax = i ;  
  
        }else if ( z[i] < min ) {  
            min = z[ i ] ;  
            imin = i ;  
        }  
    }  
    cout << "max = " << max ;  
    cout << "min = " << min ;  
}
```

One dimensional array

ตัวอย่าง 3

```
//C++ method of passing Array to function is
//passing by reference
void GetData( int A [120] ) ;
{
    int i ;
    for ( i = 0 ; i < 120 ; i++)
    {
        cin >>A[ i ] ;
    }
}
```

One dimensional array

ตัวอย่าง 4

//C/C++ does not allow to return array data
//so this function is compile time error.

```
int GetData( int A [120] ) ;  
{  
    int i ;  
    for ( i = 0 ; i < 120 ; i++)  
    {  
        cin >>A[ i ] ;  
    }  
    return A  
}
```

One dimensional array

ตัวอย่าง 3

```
void GetData(int A[ ],int n );
void main( )
{
    int z[120] y[10], i ;
    int max , min ;
    int imax , imin ;

    GetData ( Z , 120 ) ;
    GetData ( y , 10 ) ;

    max = z[0] ; min = z[0];
    imax = 0;    imin = 0;
```

```
    for ( i = 1 ; i < 120 ; i++)
    {
        if (z[ i ] > max ) {
            max = z[ i ];
            imax = i ;

        }else if ( z[i] < min ) {
            min = z[ i ];
            imin = i ;
        }
    }

    cout << "max = " << max ;
    cout << "min = " << min ;
}
```

One dimensional array

ตัวอย่าง 3

```
//C++ method of passing Array to function is  
//passing by reference  
void GetData( int A [ ] , int n ) ;  
{  
    int i ;  
    for ( i = 0 ; i < n ; i++)  
    {  
        cin >> A[ i ] ;  
    }  
}
```

One dimensional array

ตัวอย่าง 3

จงเขียนโปรแกรมรับข้อมูลจำนวนเต็ม Z ขนาด 120
ตัว และหาค่าสูงสุดและต่ำสุด รวมทั้งหาตำแหน่งของ
ค่าสูงและต่ำสุด

One dimensional array

ตัวอย่าง 3 (cont)

```
void main( )
{
    int z[120] , i ;
    int max , min ;
    int imax , imin ;
    for ( i = 0 ; i < 120 ; i++)
    {
        cin >> z[ i ] ;
    }
    max = z[0] ; min = z[0];
    imax = 0;    imin = 0;
```

```
    for ( i = 1 ; i < 120 ; i++)
    {
        if (z[ i ] > max ) {
            max = z[ i ];
            imax = i ;

        }else if ( z[i] < min ) {
            min = z[ i ];
            imin = i ;
        }
    }
    cout << "max = " << max ;
    cout << "min = " << min ;
}
```

One dimensional array

ตัวอย่าง 4 (cont)

```
void main( )
{
    int z[120] , i ;
    int max , min ;
    int imax , imin ;
    for ( i = 0 ; i < 120 ; i++)
    {
        cin >> z[ i ] ;
    }
    max = z[0] ; min = z[0];
    imax = 0;    imin = 0;
```

```
    for ( i = 1 ; i < 120 ; i++)
    {
        if (z[ i ] > max ) {
            max = z[ i ];
            imax = i ;
        }
        if ( z[i] < min ) {
            min = z[ i ];
            imin = i ;
        }
    }
    cout << "max = " << max ;
    cout << "min = " << min ;
}
```

```
#include<iostream.h>
```

```
float celsius_to_fahrenheit(float celsius);
```

```
int main()
```

```
{  
    float fahrenheit;  
    float celsius = 22.5;
```

```
C:\My Documents\sc107\files>ct  
22.5 C = 72.5 F
```

```
    fahrenheit = celsius_to_fahrenheit(celsius);  
    cout << celsius << " C = " << fahrenheit << " F\n";
```

```
    return 0;  
}
```

```
float celsius_to_fahrenheit(float celsius)
```

```
{  
    float fahr = ( celsius * (9.0/5.0) + 32.0 );  
    return fahr ;  
}
```

```
#include<iostream.h>
```

```
float celsius_to_fahrenheit( float celsius );
```

```
int main()
```

```
{  
    float fahrenheit;  
    float celsius = 22.5;
```

```
C:\My Documents\sc107\files>ct  
22.5 C = 72.5 F
```

```
    fahrenheit = celsius_to_fahrenheit( celsius );  
    cout << celsius << " C = " << fahrenheit << " F\n";
```

```
    return 0;  
}
```

```
float celsius_to_fahrenheit( float celsius )
```

```
{  
    return ( celsius * (9.0/5.0) + 32.0 );  
}
```

```
#include<iostream.h>
int celsius_to_fahrenheit( float celsius );
```

```
int main()
{
    float fahrenheit;
    float celsius = 22.5;
```

```
C:\My Documents\sc107\
22.5 C = 72 F
```

```
fahrenheit = celsius_to_fahrenheit( celsius );
```

```
cout << celsius << " C = " << fahrenheit << " F\n";
```

```
return 0;
}
```

```
int celsius_to_fahrenheit( float celsius )
{
    return (celsius * (9.0/5.0) + 32.0 );
}
```

```
#include<iostream.h>
```

```
celsius_to_fahrenheit(float celsius);
```

```
int main()
```

```
{
```

```
    float fahrenheit;
```

```
    float celsius = 22.5;
```

```
    fahrenheit = celsius_to_fahrenheit(celsius);
```

```
    cout << celsius << " C = " << fahrenheit << " F\n";
```

```
    return 0;
```

```
}
```

```
celsius_to_fahrenheit(float celsius)
```

```
{
```

```
    return(celsius * (9.0/5.0) + 32.0);
```

```
}
```

```
C:\My Documents\sc107\file  
22.5 C = 72 F
```

```
#include<iostream.h>
```

```
float celsius_to_fahrenheit(float celsius);
```

```
int main()
```

```
{
```

```
    int fahrenheit ;
```

```
    float celsius = 22.5;
```

```
    fahrenheit = celsius_to_fahrenheit(celsius);
```

```
    cout << celsius << " C = " << fahrenheit << " F\n";
```

```
    return 0;
```

```
}
```

```
float celsius_to_fahrenheit(float celsius)
```

```
{
```

```
    return(celsius * (9.0/5.0) + 32.0);
```

```
}
```

```
C:\My Documents\sc107\file
```

```
22.5 C = 72 F
```



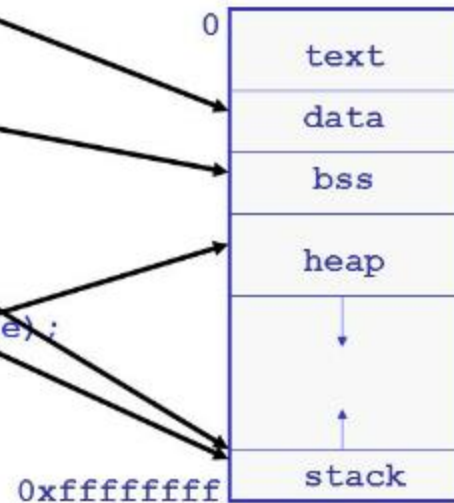
Recommend about “return”

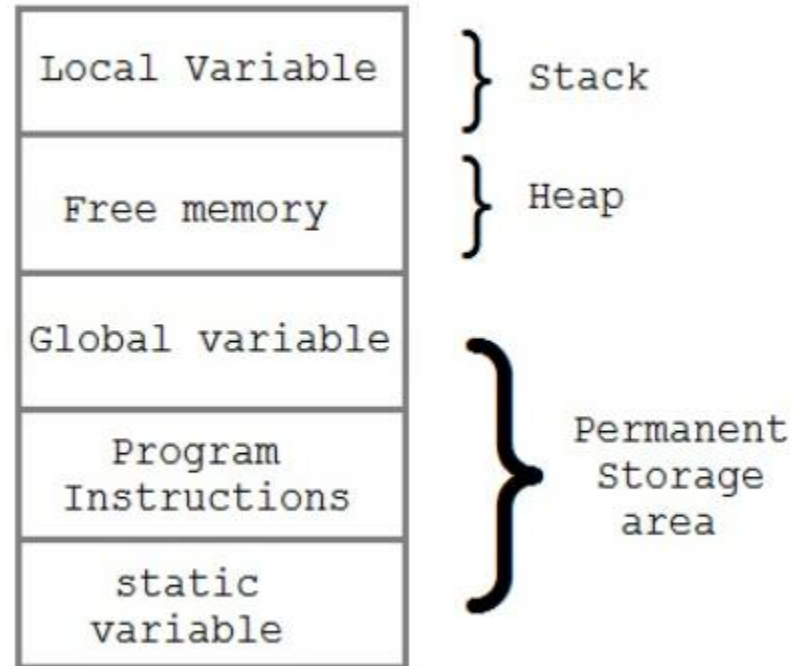
- ❑ The return statement does not require that the value being returned be placed in parentheses.
- ❑ A function can return only one value using return statement
- ❑ When a return statement is encountered, the function will exit and return the value specified.



Example

```
char *string = "hello";  
  
int iSize;  
  
char *f (int x)  
{  
    char *p;  
  
    iSize = 8;  
    p = (char *)malloc (iSize);  
  
    return p;  
}
```









C/C++ Structure Type & Function

□ C/C++ support both pass by value & pass by reference





Array with Function in C/C++

- Primitive type : int , float , long , double ,
char
- User define type : structure
- Array is using “pass by reference”.

Pass array to function

```
8
9 #include <iostream>
10 using namespace std;
11 void updateArray( int data[ ],int n )
12 {
13     for(int i = 0 ; i < n ; i++) {
14         data[i] = data[i]+1000;
15     }
16 }
17 void showarray( int data[ ],int n )
18 {
19     for(int i = 0 ; i < n ; i++) {
20         cout << data[i] << ",";
21     }
22 }
23 int main() {
24
25     int Roll_Number[5] = {100,101,102,103,104};
26
27     cout << "\nbefore update \n";
28     showarray(Roll_Number,5);
29     updateArray(Roll_Number,3);
30     cout << "\nafter update \n";
31     showarray(Roll_Number,5);
32 }
```

before update
100,101,102,103,104,
after update
1100,1101,1102,103,104,

Return data with array in C/C++

- ❑ C/C++ not allow the code return data in array form

Run

Debug

Stop

Share

main.cpp

```
1  #include <iostream>
2
3  using namespace std;
4
5  int * dosomething6() {
6      int x[5]={0};
7
8      for(int i=0 ; i < 5 ;i++)
9          x[i] = i;
10     return x;
11 }
12
13 int main()
14 {
15     int x[5] ;
16     x = dosomething6();
17     for(int i=0 ; i < 5 ;i++)
18         cout << x[i] << " ";
19     return 0;
20 }
21
```

input

Compilation failed due to following error(s).

```
main.cpp: In function 'int* dosomething6()':
```

```
10 |     return x;
    |         ^
```

```
main.cpp:6:9: note: declared here
```

```
6 | int x[5]={0};
  |      ^
```

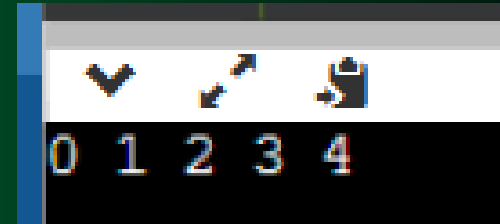
```
main.cpp: In function 'int main()':
```

```
16 | x = dosomething6();  
    ^NNNNNNNNNNNNNNNNNNNN
```

Getting value in array using passing parameter

main.cpp

```
1  //
2  #include <iostream>
3
4  using namespace std;
5
6  void dosomething7(int x[],int n) {
7      for(int i=0 ; i < n ;i++)
8          x[i] = i;
9  }
10 int main()
11 {
12     int x[5] = {0};
13     dosomething7(x,5);
14     for(int i=0 ; i < 5 ;i++)
15         cout << x[i] << " ";
16     return 0;
17 }
18
```





main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  struct employee {
5      char name[100];
6      int age;
7      float salary;
8      char department[50];
9  };
10
11  // This function takes structure variable as parameter
12  void printEmployeeDetails(struct employee emp){
13      cout << "\n*** Employee Details ***\n";
14      cout << "Name : " << emp.name << "\nAge : " << emp.age << "\nSalary : "
15          << emp.salary << "\nDepartment : " << emp.department;
16  }
17  int main(){
18      struct employee manager ;
19
20      printf("Enter Name, Age, Salary and Department of Employee\n");
21      // Assigning data to members of structure variable
22      cin >> manager.name >> manager.age >> manager.salary >> manager.department;
23
24      // Passing structure variable to function
25      printEmployeeDetails(manager);
26  }
```

Enter Name, Age, Salary and Department of Employee
apirak
30
1000.5
ram

*** Employee Details ***
Name : apirak
Age : 30
Salary : 1000.5
Department : ram

```

7
8
9 #include <iostream>
10 using namespace std;
11 struct myStructure {
12     int x;
13     int y;
14 };
15
16 void swap(myStructure &ms)
17 {
18     int temp = ms.x;
19     ms.x = ms.y;
20     ms.y = temp;
21 }
22
23 int main()
24 {
25     myStructure ms = { 50, 60 };
26
27     cout<<"\n\nBefore swap, inside main():\n";
28     cout<<"x = "<<ms.x<<"\ty = "<<ms.y;
29
30     swap(ms);
31
32     cout<<"\n\nAfter swap, inside main():\n";
33     cout<<"x = "<<ms.x<<"\ty = "<<ms.y;
34     cout<<endl;
35     return 0;
36 }
37

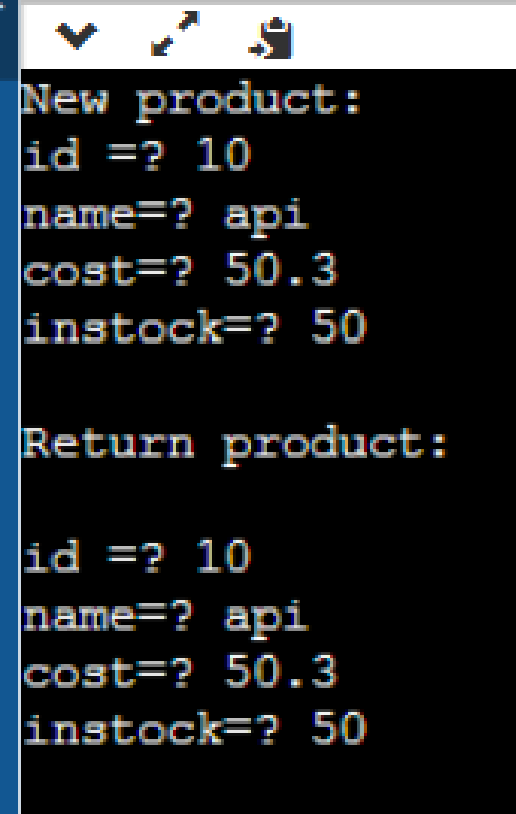
```

Before swap, inside main():
x = 50 y = 60

After swap, inside main():
x = 60 y = 50

Function return Structure Type in C/C++

```
4 struct product {  
5     char id[10];  
6     char name[30];  
7     float cost;  
8     int instock;  
9 };  
10  
11 struct product getNewProduct() {  
12     struct product p;  
13     cout << "New product: " << endl;  
14     cout << "id =? "; cin >> p.id;  
15     cout << "name=? "; cin >> p.name;  
16     cout << "cost=? "; cin >> p.cost;  
17     cout << "instock=? "; cin >> p.instock;  
18     return p;  
19 }  
20 int main() {  
21     struct product newProduct = getNewProduct();  
22     cout << "\nReturn product: " << endl;  
23     cout << "\nid =? " << newProduct.id;  
24     cout << "\nname=? " << newProduct.name;  
25     cout << "\ncost=? " << newProduct.cost;  
26     cout << "\ninstock=? " << newProduct.instock;  
27 }  
28
```



```
New product:  
id =? 10  
name=? api  
cost=? 50.3  
instock=? 50  
  
Return product:  
  
id =? 10  
name=? api  
cost=? 50.3  
instock=? 50
```

Menu

1. Add item to Cart
2. Sell Product
3. Check Stock
4. End Program

Please select 1 / 2 / 3/ 4 =?

Ex 2 จงใช้โครงสร้างข้อมูลแบบ record

เงินเดือนของพนักงาน XYZ ขึ้นอยู่กับ ฐานเงินเดือน ที่เท่ากันทุกคน คือ 3000 บาท บวกกับเปอร์เซ็นต์ของฐานเงินเดือนตาม ชนิดของงาน จำนวนปีที่ทำงาน ระดับความรู้ ความเอาใจใส่งาน

$$\text{รายได้ (ic)} = \text{เงินเดือน} + \text{ฐานเงินเดือน} * (\%$$

ชนิดของงาน	%	ระดับความรู้	%
1	0	1 (ป.ว.ช)	0
2	5	2 (ป.ว.ส)	5
3	15	3 (ป.ตรี)	15

Ex 2 จงใช้โครงสร้างข้อมูลแบบ record (ต่อ)

จงเขียนโปรแกรมรับตัวเลขรหัสของ 4 รายการข้างต้นของพนักงานแต่ละคน เพื่อคำนวณเงินเดือน จำนวน 120 คน

จำนวนปีที่ทำงาน	%	ความเอาใจใส่งาน	%
0 - 10	5	0 (ไม่ดี)	0
แต่ละปีถัดไป	4	1 (ดี)	10
		2 (ดีมาก)	25

Ex 2 จงใช้โครงสร้างข้อมูลแบบ record

เงินเดือนของพนักงาน XYZ ขึ้นอยู่กับ ฐานเงินเดือน ที่เท่ากันทุกคน คือ 3000 บาท บวกกับเปอร์เซ็นต์ของฐานเงินเดือนตาม ชนิดของงาน จำนวนปีที่ทำงาน ระดับความรู้ ความเอาใจใส่งาน

รายได้ (ic) = เงินเดือน + ฐานเงินเดือน * (% ชนิดงาน)

+ ฐานเงินเดือน * (%จำนวนปีที่ทำงาน)

+ ฐานเงินเดือน * (%ความเอาใจใส่งาน)



แบบฝึกหัด 2

จงเขียน โปรแกรมต้องการนำทะเบียนลูกค้า(customer) ของ
สหกรณ์ออมทรัพย์รามคำแหง ที่รองรับการจัดเก็บข้อมูลไม่
เกิน 500 คน ทะเบียนลูกค้าประกอบด้วย รหัสบัญชี(id)
ชื่อบัญชี(name) ที่อยู่(Address) และเงินฝาก(Deposit)



Menu

1. Open a new Account
2. Deposit
3. Withdraw
4. Compound interest
5. Quit

Please select 1 / 2 / 3 / 4 / 5 =?



เลือก 1 เป็นการเปิดบัญชีใหม่

ระบบจะแสดงหน้าจอให้ผู้ใส่ ป้อนรหัสบัญชี(id)

ชื่อบัญชี(name) ที่อยู่(Address) และเงินฝาก

(Deposit) โดยรหัสบัญชีต้องไม่ซ้ำกัน

เลือก 2 เป็นการฝากเงิน

โดยผู้ใช้ป้อนรหัสบัญชี(id)ที่ต้องการ ระบบจะ

ตรวจสอบเพื่อกันหาระเบียบ**ลูกค้า** เพื่อให้ผู้ใช้งาน**ป้อน**

ยอดฝาก เพื่อปรับปรุงบัญชีให้ถูกต้อง

เลือก 3 เป็นการถอนเงิน

โดยผู้ใช้ป้อนรหัสบัญชี(id)ที่ต้องการ ระบบจะตรวจสอบเพื่อ
ค้นหากระเป๋เงินลูกค้า เพื่อให้ผู้ใช้งานป้อนยอดถอนแต่ต้องมีเงินติด
บัญชีอย่างน้อย 100 บาท ระบบทำการปรับปรุงบัญชีให้ถูกต้อง

เลือก 4 เป็นการคิดดอกเบี้ยทบต้น

โดยระบบจะให้ผู้ใช้ป้อนอัตราดอกเบี้ยต่อปี(rate) ทาง
แบงค์พิมพ์ เพื่อคิดดอกเบี้ยทบต้นของทุกบัญชี ซึ่งกระทำทุกวันได้
ดอกเบี้ยเป็นรายวัน โปรแกรมจะแสดงเงินฝากเก่า ดอกเบี้ยที่
ได้รับ และเงินฝากใหม่(เงินฝากเดิม+ดอกเบี้ยที่ได้)ปรากฏบน
เงินฝากใหม่ = เงินฝากเดิม+ดอกเบี้ยที่ได้

จอภาพ : ดอกเบี้ยที่ได้ = อัตราดอกเบี้ยที่ได้ * เงินฝากเดิม



แบบฝึกหัด 2

จงเขียน โปรแกรมต้องการนำทะเบียนลูกค้า(customer) ของ
สหกรณ์ออมทรัพย์รามคำแหง เก็บในตัวแปร

แถวลำดับ 1 มิติ ทะเบียนลูกค้าประกอบด้วย รหัสบัญชี(id)
ชื่อบัญชี(name) ที่อยู่(Address) และเงินฝาก(Deposit)



Menu

1. Open a new Account
2. Deposit
3. Withdraw
4. Compound interest
5. Quit

Please select 1 / 2 / 3 / 4 / 5 =?



เลือก 1 เป็นการเปิดบัญชีใหม่

ระบบจะแสดงหน้าจอให้ผู้ใส่ ป้อนรหัสบัญชี(id)

ชื่อบัญชี(name) ที่อยู่(Address) และเงินฝาก

(Deposit) โดยรหัสบัญชีต้องไม่ซ้ำกัน

เลือก 2 เป็นการฝากเงิน

โดยผู้ใช้ป้อนรหัสบัญชี(id)ที่ต้องการ ระบบจะ

ตรวจสอบเพื่อกันหาระเบียบนลูกค้า เพื่อให้ผู้ใช้งานป้อน

ยอดฝาก เพื่อปรับปรุงบัญชีให้ถูกต้อง



เลือก 3 เป็นการถอนเงิน

โดยผู้ใช้ป้อนรหัสบัญชี(id)ที่ต้องการ ระบบจะตรวจสอบเพื่อ
ค้นหากระเป๋เงินลูกค้า เพื่อให้ผู้ใช้งานป้อนยอดถอนแต่ต้องมีเงินติด
บัญชีอย่างน้อย 100 บาท ระบบทำการปรับปรุงบัญชีให้ถูกต้อง

เลือก 4 เป็นการคิดดอกเบี้ยทบต้น

โดยระบบจะให้ผู้ใช้ป้อนอัตราดอกเบี้ยต่อปี(rate) ทาง
แอปพลิเคชัน เพื่อคิดดอกเบี้ยทบต้นของทุกบัญชี ซึ่งกระทำทุกวันได้
ดอกเบี้ยเป็นรายวัน โปรแกรมจะแสดงเงินฝากเก่า ดอกเบี้ยที่
ได้รับ และเงินฝากใหม่ปรากฏบนจอภาพ

เลือก 5 เป็นการจบการทำงานซ้ำ



Array 2 dimension Passing In C/C++



ตัวอย่าง แบบ top down

จงเขียนโปรแกรม**รับข้อมูล**ลงอาร์เรย์ A , B ขนาด 20x20
และ**หาผลคูณ**ของอาร์เรย์ C โดย

$$C = A \times B$$

สูตรการคูณ คือ $C_{ij} = \sum a_{ik} b_{kj}$

เมื่อ $1 \leq i \leq 20$ และ $1 \leq j \leq 20$



Break down the problem

1. Get data both A and B
2. Calculate $C = A * B$;
3. Show C

```

1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  void GetData ( int Z [3 ] [3] )
6  {
7      int i , j ;
8
9      for ( i = 0 ; i < 3 ; i++ )
10     {
11         for ( j = 0 ; j < 3 ; j++ )
12         {
13             //cout << " row = " << i << "column " << j ;
14             //cin >> Z[ i ] [ j ] ;
15             // Get a random number
16             Z[ i ] [ j ] = rand()%10;
17             cout << setw(5) << Z[ i ] [ j ] << " ";
18         }
19         cout << endl;
20     }
21 }
22
23 void MultiplyMatrix ( int W[ 3 ] [3 ],
24                       int X[3][3],int Z[3][3 ] )
25 {
26     int i , j , k ,sum ;
27     for ( i = 0 ; i < 3 ; i++ )
28     {
29         for ( j = 0 ; j < 3 ; j++ )
30         {
31             sum = 0 ;
32             for ( k = 0 ; k < 3 ; k++ )
33             {
34                 sum =sum + W[ i ] [ k ] * X [ k ] [ j ] ;
35             }
36             Z[ i ] [ j ] = sum ;
37         }
38     }
39 }
40

```

```

40
41 void PrintMatrix( int Z [3 ] [3] )
42 {
43     int i , j ;
44     cout << "value of Matrix C \n" ;
45     for ( i = 0 ; i < 3 ; i++ )
46     {
47         for ( j = 0 ; j < 3 ; j++ )
48         {
49             cout << setw(5) << Z[ i ] [ j ] << " ";
50         }
51         cout << endl;
52     }
53 }
54 int main( )
55 {
56     int A[ 3 ] [ 3 ] ;
57     int B[ 3 ] [3] ;
58     int C[ 3 ] [ 3 ] ;
59     int i , j ;
60
61     cout << "input DATA of Matrix A \n" ;
62     GetData ( A ) ;
63     cout << "input DATA of Matrix B \n" ;
64     GetData ( B ) ;
65     MultiplyMatrix ( A, B , C ) ;
66     PrintMatrix(C);
67 }
68

```



Apirak :

```

1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4  int main( )
5  {
6      int A[ 4 ] [ 5 ] ;
7      int i , j ;
8      int max , col_max , row_max ;
9
10     srand((unsigned) time(NULL));
11     cout << "input DATA of Matrix A \n" ;
12     for ( i = 0 ; i < 4 ; i++ )
13     {
14         for ( j = 0 ; j < 5 ; j++ )
15         {
16             //cout <<"row = " << i << " column " << j << " " ;
17             //cin >> A[ i ] [ j ] ;
18             A[ i ] [ j ] = rand()%15;
19             //cout << "A[" << i << "]" [" << j << "]" " << A[ i ] [ j ] << endl;
20             cout << setw(5) << A[ i ] [ j ] << " ";
21         }
22         cout << endl;
23     }
24

```

```

/tmp/CDh1D0GEgA.o
input DATA of Matrix A
    7      8      11      10      11
    2      4       2       2       3
    1     14       8       9      14
    5      5     14       6       5
max value = 14
in column = 1
in row  =  2

```

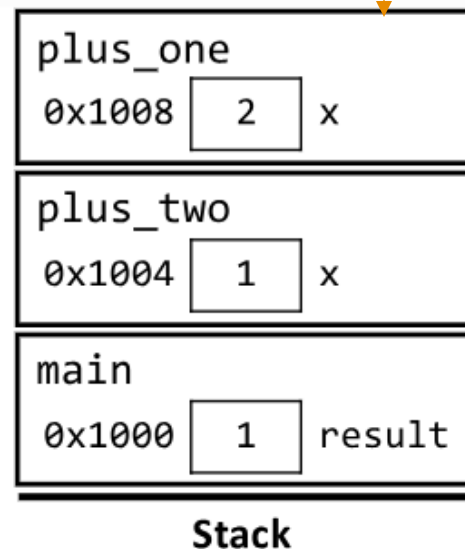
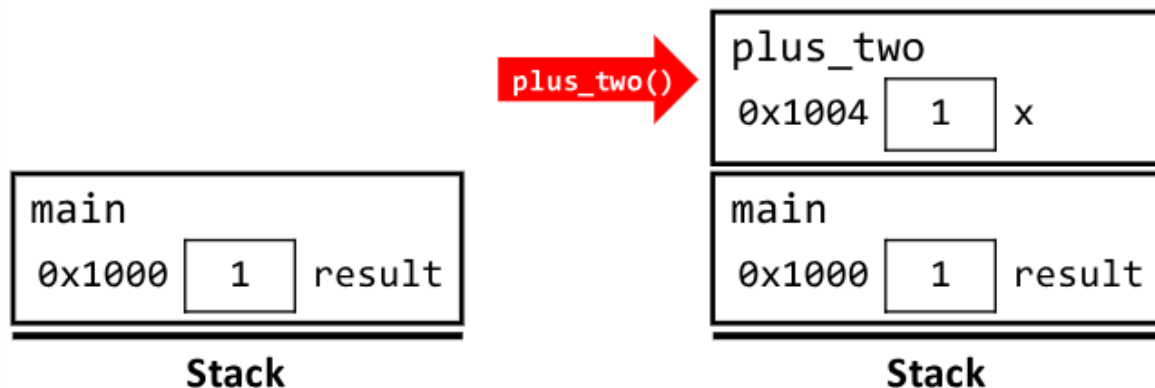
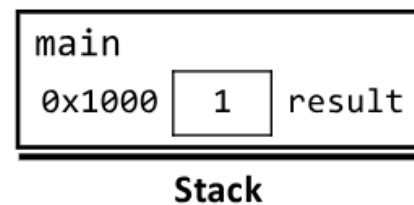
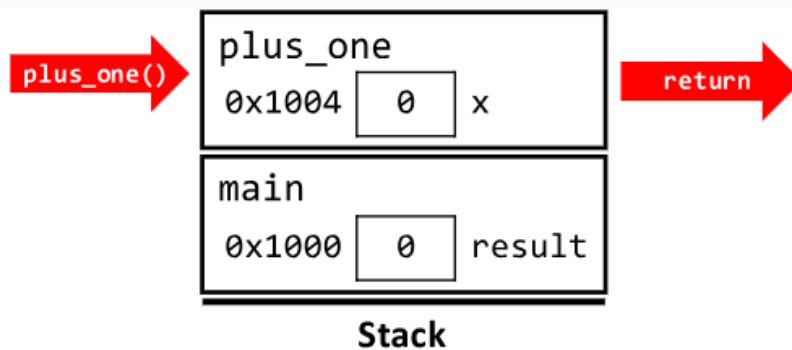
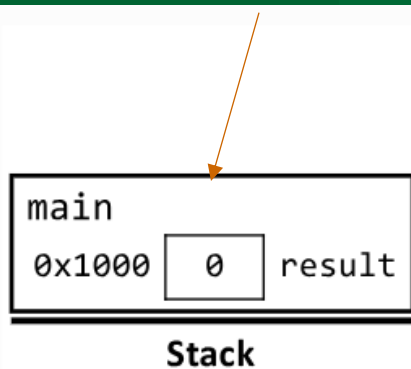
Execution flows (4)

OS call

```
int plus_one(int x) {  
    return x + 1;  
}
```

```
int plus_two(int x) {  
    return plus_one(x + 1);  
}
```

```
int main() {  
    int result = 0;  
    result = plus_one(0);  
    result = plus_two(result);  
    cout << result;  
} // print
```



return array (1)

main.cpp

```
1 #include <iostream.h>
2
3 //using namespace std;
4
5 int *dosomething6() {
6     int x[5]={0};
7     for(int i=0 ; i < 5 ;i++)
8         x[i] = i;
9     return x;
10 }
11 int main()
12 {
13     int *x;
14     x = dosomething6();
15     for(int i=0 ; i < 5 ;i++)
16         cout << x[i] << " ";
17     return 0;
18 }
```

main.cpp

```
1 #include <iostream>
2
3 using namespace std;
4
5 int *dosomething6() {
6     int x[5]={0};
7     for(int i=0 ; i < 5 ;i++)
8         x[i] = i;
9     return x;
10 }
11 int main()
12 {
13     int x[5] ;
14     x = dosomething6();
15     for(int i=0 ; i < 5 ;i++)
16         cout << x[i] << " ";
17     return 0;
18 }
19
```

Compilation failed due to following error(s).

```
main.cpp: In function ‘int* dosomething6()’:  
main.cpp:9:12: warning: address of local variable ‘x’ returned [-Wreturn-local-addr]  
    9 |         return x;  
      |             ^  
  
main.cpp:6:9: note: declared here  
    6 |     int x[5]={0};  
      |       ^  
  
main.cpp: In function ‘int main()’:  
main.cpp:14:7: error: incompatible types in assignment of ‘int**’ to ‘int [5]’  
   14 |     x = dosomething6();  
      |           ~^~
```

Return local array

```
main.cpp
1  #include <iostream>
2
3  using namespace std;
4
5  int *dosomething6() {
6      int x[5]={0};
7      for(int i=0 ; i < 5 ;i++)
8          x[i] = i;
9      return x;
10 }
11 int main()
12 {
13     int x[5] ;
14     x = dosomething6();
15     for(int i=0 ; i < 5 ;i++)
16         cout << x[i] << " ";
17     return 0;
18 }
19
20
```

```
main.cpp
1  #include <iostream>
2
3  using namespace std;
4
5  void dosomething7(int x[],int n) {
6      for(int i=0 ; i < n ;i++)
7          x[i] = i;
8  }
9  int main()
10 {
11     int x[5] ;
12     dosomething7(x,5);
13     for(int i=0 ; i < 5 ;i++)
14         cout << x[i] << " ";
15     return 0;
16 }
17
```